Design Microsoft Teams

> 📁 **This guide has been split into multiple pages for better navigation.**
>
> **→ Go to the new guide**

Quick Links

Page	Description
Overview	Architecture, components, clarifying questions
Chat & Messaging	Messaging flow, sync, presence, notifications
Video & Calls	WebRTC, SFU, recording, screen sharing
File Sharing	Upload flow, SAS tokens, CDN
Databases	Cosmos DB, SQL, Redis, Blob, Search
Interview Tips	Trade-offs, how to explain, Q&A

Original Content (kept for reference)

Problem Statement

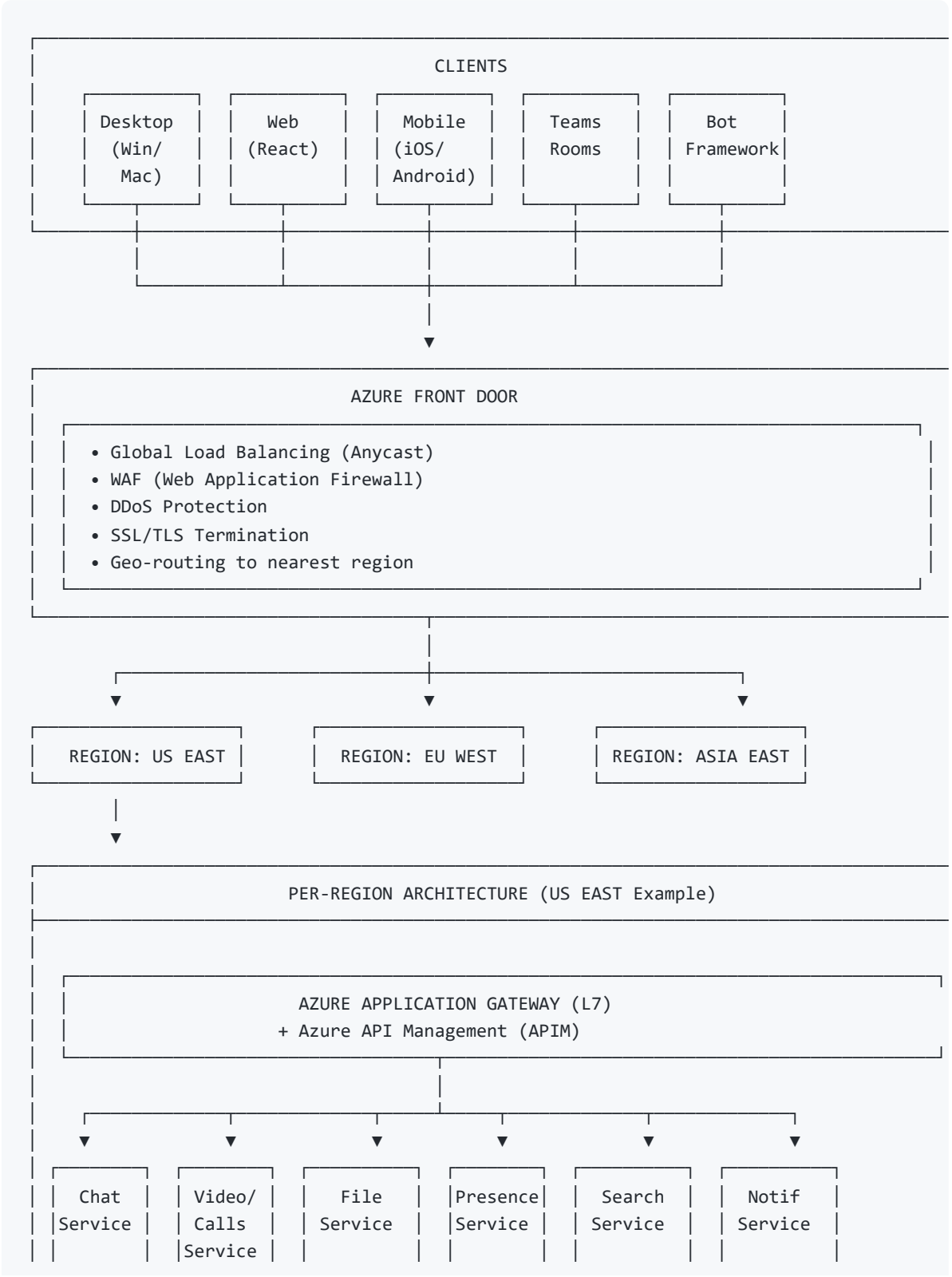
Design a real-time collaboration platform like Microsoft Teams that supports messaging, video calls, file sharing, and integrations for millions of concurrent users.

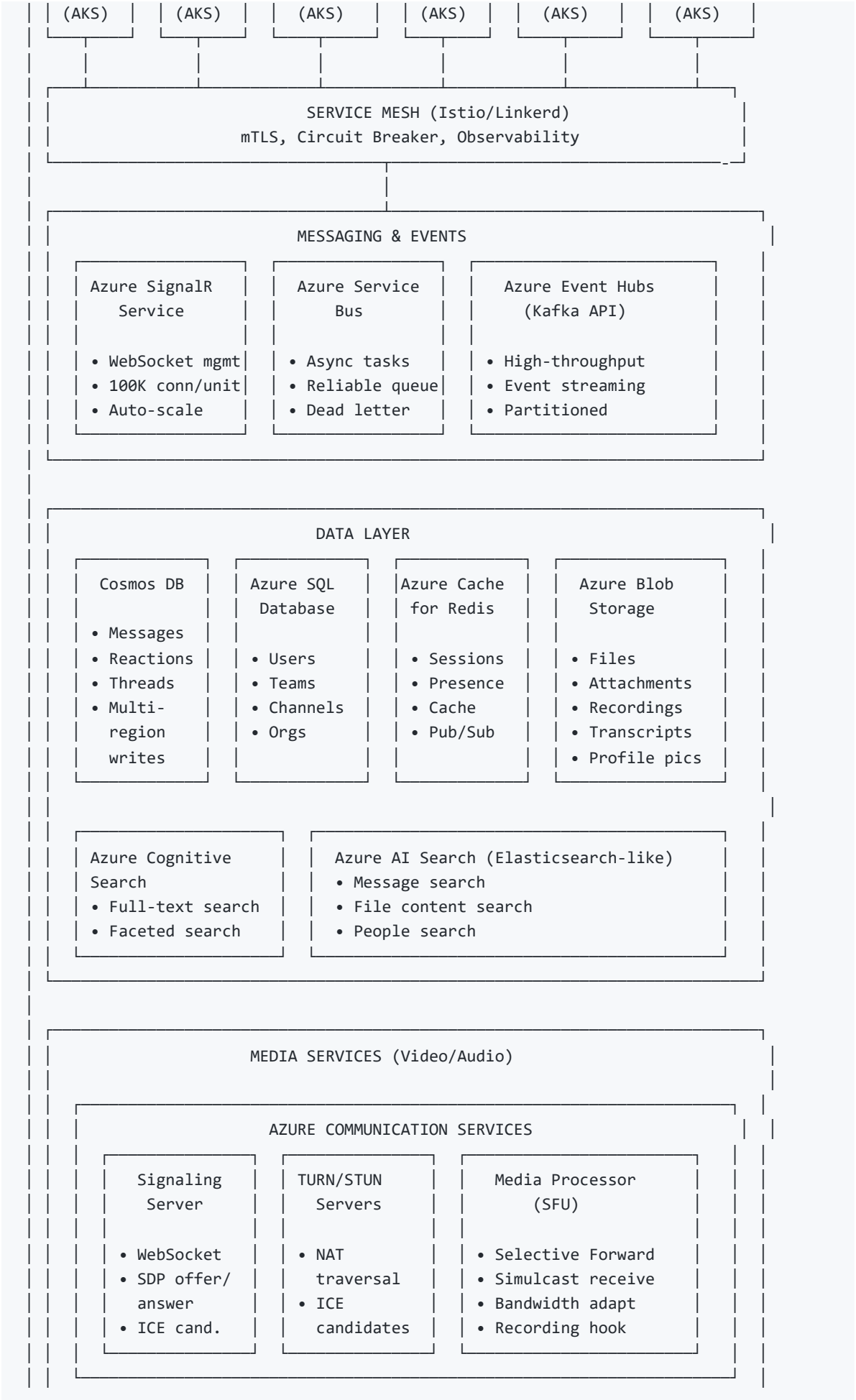
Clarifying Questions to Ask

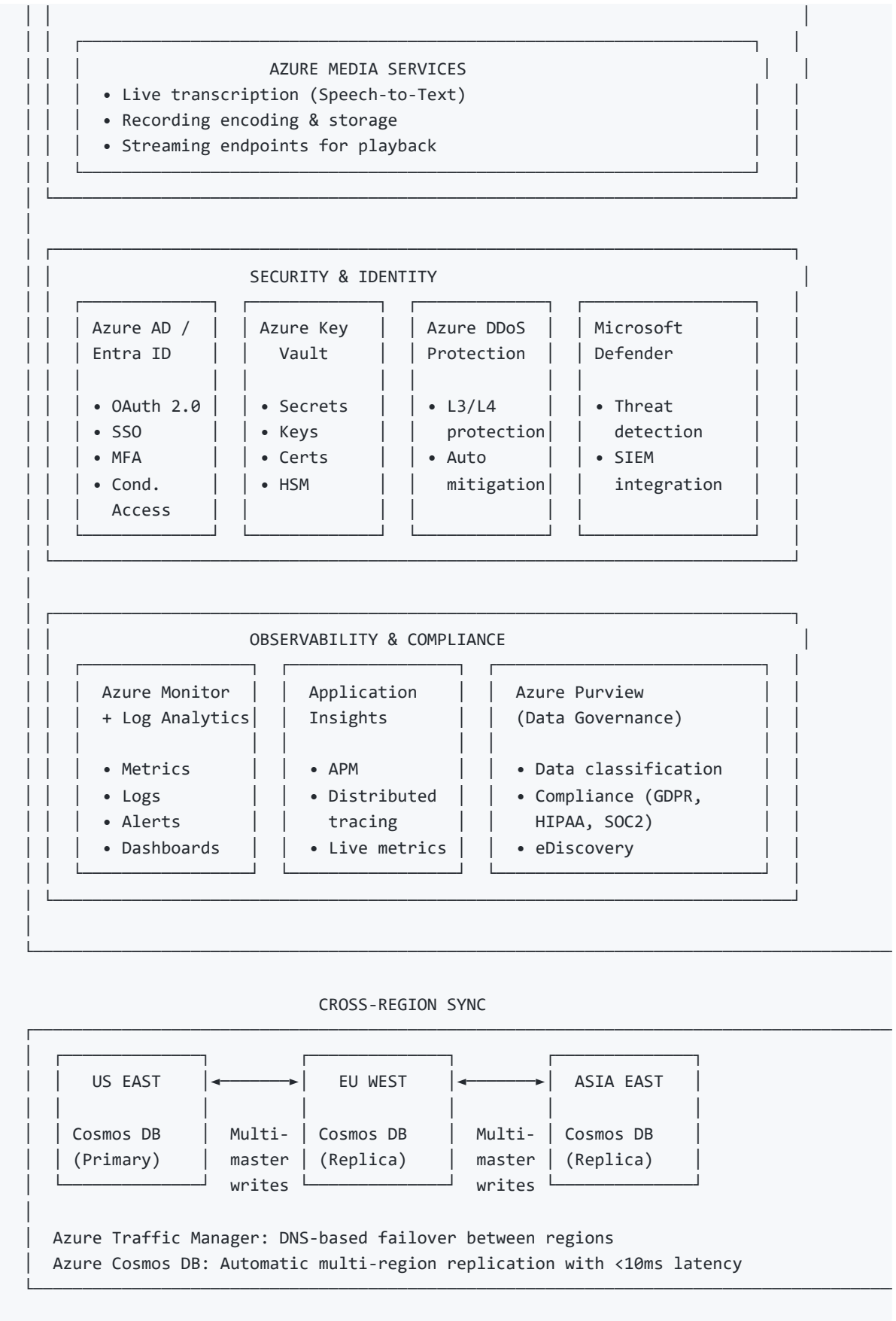
1. **Scale**: How many daily active users? (e.g., 300M DAU)
2. **Features**: Which features to prioritize? (chat, video, file sharing, screen share)
3. **Message retention**: How long to store messages? (compliance requirements)
4. **Availability**: What's the target uptime? (99.99%?)
5. **Latency**: Max acceptable latency for messages? (<100ms?)
6. **Offline support**: Should users see messages when back online?
7. **Enterprise features**: Do we need admin controls, compliance, DLP?

8. **Call size**: Max participants in a call? (1:1, group up to 1000?)
9. **Recording**: Do calls need to be recorded and stored?
10. **PSTN**: Integration with phone networks?

Complete Azure Architecture




(AKS)
(AKS)
(AKS)
(AKS)
(AKS)
(AKS)
SERVICE MESH (Istio/Linkerd)
mTLS, Circuit Breaker, Observability
MESSAGING & EVENTS
Azure SignalR Service
• WebSocket mgmt
• 100K conn/unit
• Auto-scale
Azure Service Bus
• Async tasks
• Reliable queue
• Dead letter
Azure Event Hubs (Kafka API)
• High-throughput
• Event streaming
• Partitioned
DATA LAYER
Cosmos DB
• Messages
• Reactions
• Threads
• Multi-region writes
Azure SQL Database
• Users
• Teams
• Channels
• Orgs
Azure Cache for Redis
• Sessions
• Presence
• Cache
• Pub/Sub
Azure Blob Storage
• Files
• Attachments
• Recordings
• Transcripts
• Profile pics
Azure Cognitive Search
• Full-text search
• Faceted search
Azure AI Search (Elasticsearch-like)
• Message search
• File content search
• People search
MEDIA SERVICES (Video/Audio)
AZURE COMMUNICATION SERVICES
Signaling Server
• WebSocket
• SDP offer/answer
• ICE cand.
TURN/STUN Servers
• NAT traversal
• ICE candidates
Media Processor (SFU)
• Selective Forward
• Simulcast receive
• Bandwidth adapt
• Recording hook


AZURE MEDIA SERVICES
• Live transcription (Speech-to-Text)
• Recording encoding & storage
• Streaming endpoints for playback
SECURITY & IDENTITY
Azure AD / Entra ID
• OAuth 2.0
• SSO
• MFA
• Cond. Access
Azure Key Vault
• Secrets
• Keys
• Certs
• HSM
Azure DDoS Protection
• L3/L4 protection
• Auto mitigation
Microsoft Defender
• Threat detection
• SIEM integration
OBSERVABILITY & COMPLIANCE
Azure Monitor + Log Analytics
• Metrics
• Logs
• Alerts
• Dashboards
Application Insights
• APM
• Distributed tracing
• Live metrics
Azure Purview (Data Governance)
• Data classification
• Compliance (GDPR, HIPAA, SOC2)
• eDiscovery
CROSS-REGION SYNC
US EAST
Cosmos DB (Primary)
Multi-master writes
EU WEST
Cosmos DB (Replica)
Multi-master writes
ASIA EAST
Cosmos DB (Replica)
Azure Traffic Manager: DNS-based failover between regions
Azure Cosmos DB: Automatic multi-region replication with <10ms latency

API Design

Chat APIs

Method	Endpoint	Description
`POST`	`/api/v1/messages`	Send a message to a channel or chat
`GET`	`/api/v1/messages?chatId={id}&limit=50&before={cursor}`	Fetch messages with cursor pagination
`PUT`	`/api/v1/messages/{messageId}`	Edit a message
`DELETE`	`/api/v1/messages/{messageId}`	Delete a message
`POST`	`/api/v1/messages/{messageId}/reactions`	Add a reaction

Presence APIs

Method	Endpoint	Description
`PUT`	`/api/v1/presence`	Update user presence status
`GET`	`/api/v1/presence/batch?userIds={ids}`	Get presence for multiple users

Call/Meeting APIs

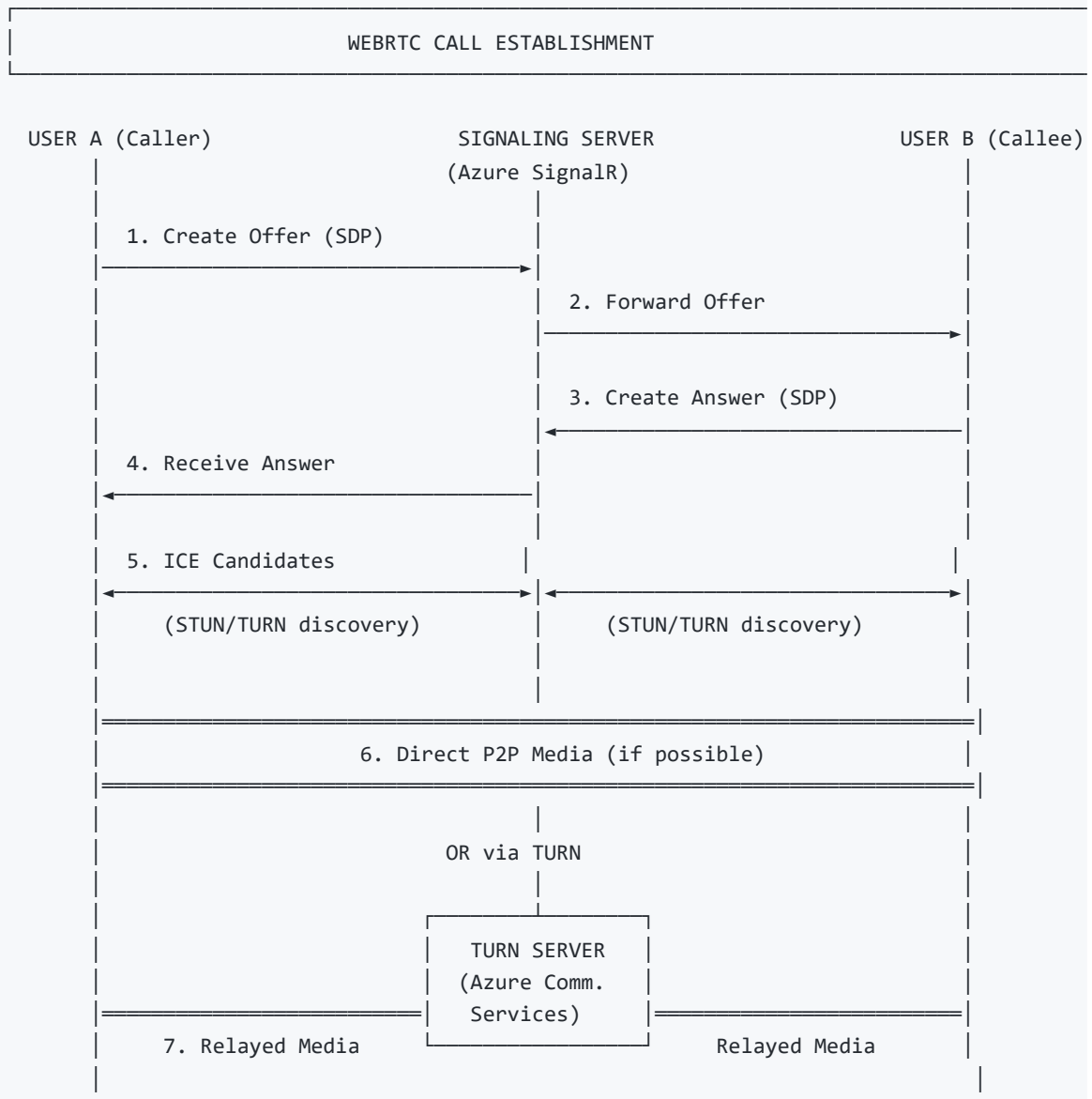
Method	Endpoint	Description
`POST`	`/api/v1/calls`	Initiate a call
`POST`	`/api/v1/calls/{callId}/join`	Join an existing call
`POST`	`/api/v1/calls/{callId}/leave`	Leave a call
`POST`	`/api/v1/meetings`	Schedule a meeting

WebSocket Events

Event	Direction	Payload
`message.new`	Server → Client	`{ chatId, message, sender, timestamp }`
`message.typing`	Client → Server	`{ chatId, userId }`
`presence.changed`	Server → Client	`{ userId, status }`
`call.incoming`	Server → Client	`{ callId, caller, type }`

Video/Calls Deep Dive

WebRTC Call Flow

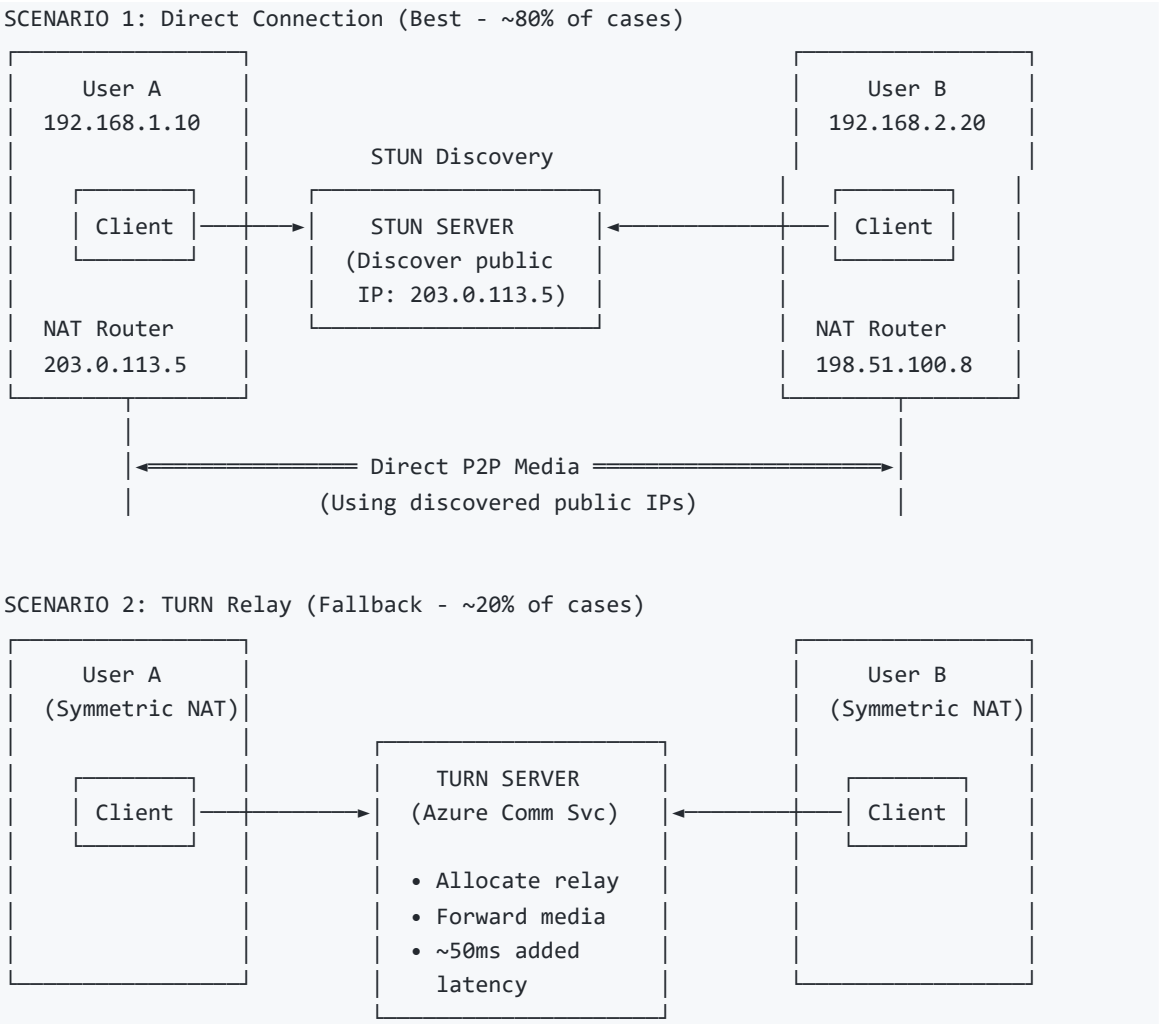


SDP (Session Description Protocol) contains:

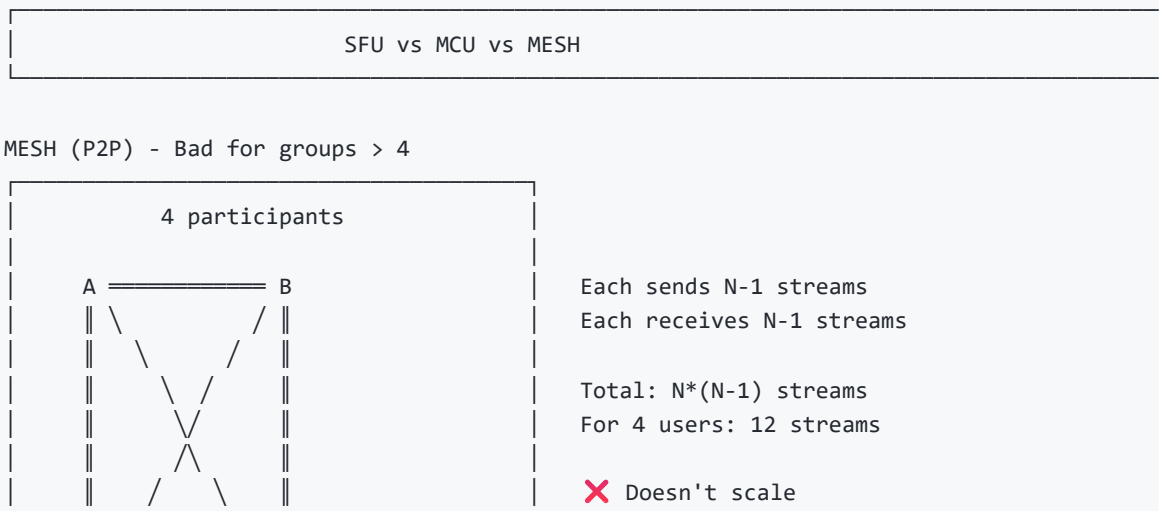
- Media types (audio, video, screen share)
- Codecs supported (VP8, VP9, H.264, Opus)
- ICE candidates (connection paths)
- DTLS fingerprints (encryption)

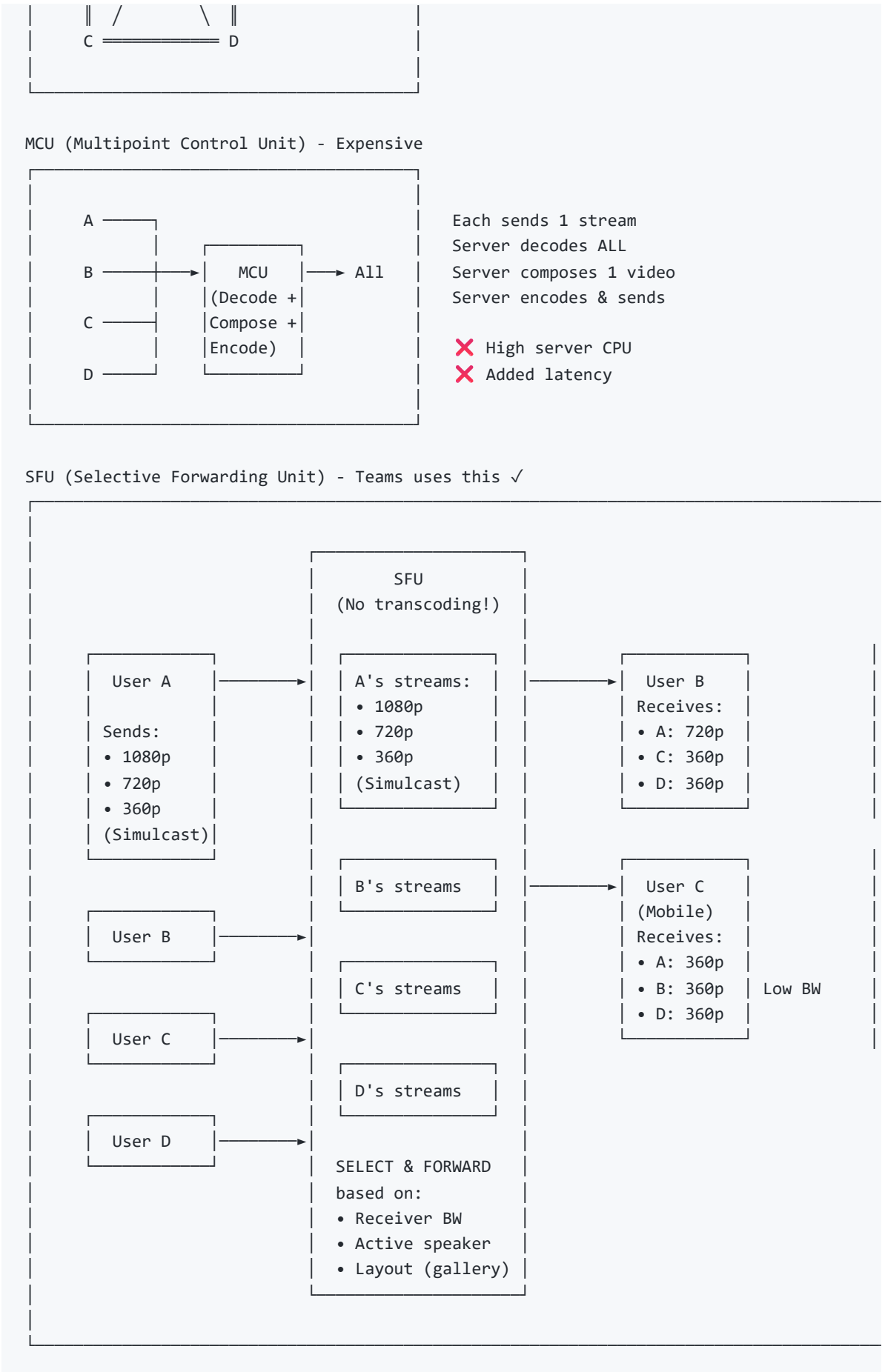
STUN/TURN NAT Traversal





SFU Architecture (Selective Forwarding Unit)

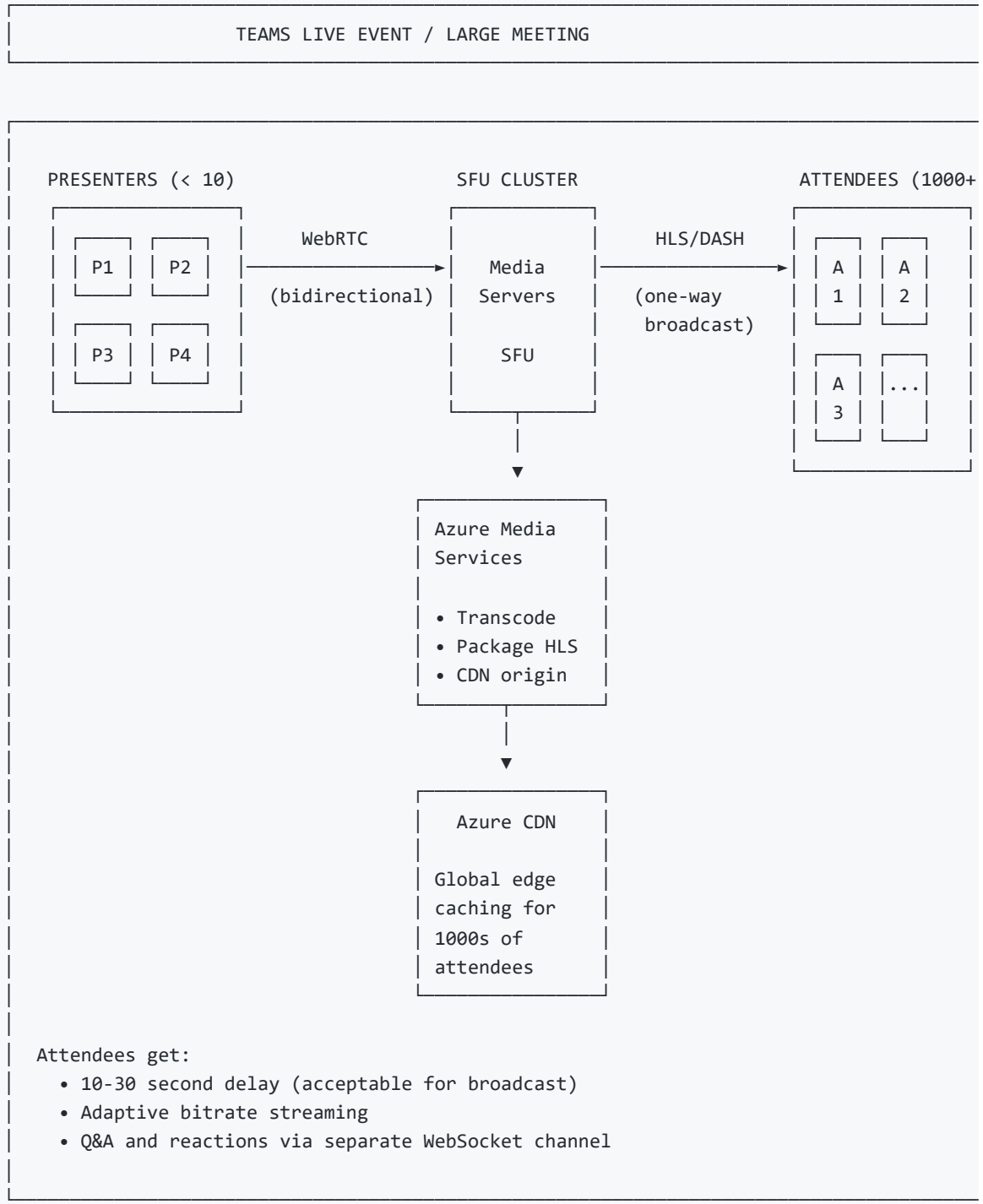



C ═══ D
MCU (Multipoint Control Unit) - Expensive
A
B
C
D
MCU
(Decode +
Compose +
Encode)
All
Each sends 1 stream
Server decodes ALL
Server composes 1 video
Server encodes & sends
High server CPU
Added latency
SFU (Selective Forwarding Unit) - Teams uses this ✓
SFU
(No transcoding!)
User A
Sends:
• 1080p
• 720p
• 360p
(Simulcast)
User B
User C
User D
A's streams:
• 1080p
• 720p
• 360p
(Simulcast)
B's streams
C's streams
D's streams
SELECT & FORWARD
based on:
• Receiver BW
• Active speaker
• Layout (gallery)
User B
Receives:
• A: 720p
• C: 360p
• D: 360p
User C
(Mobile)
Receives:
• A: 360p
• B: 360p
• D: 360p
Low BW
SIMULCAST: Sender encodes 3 quality layers
• High: 1080p @ 2.5 Mbps

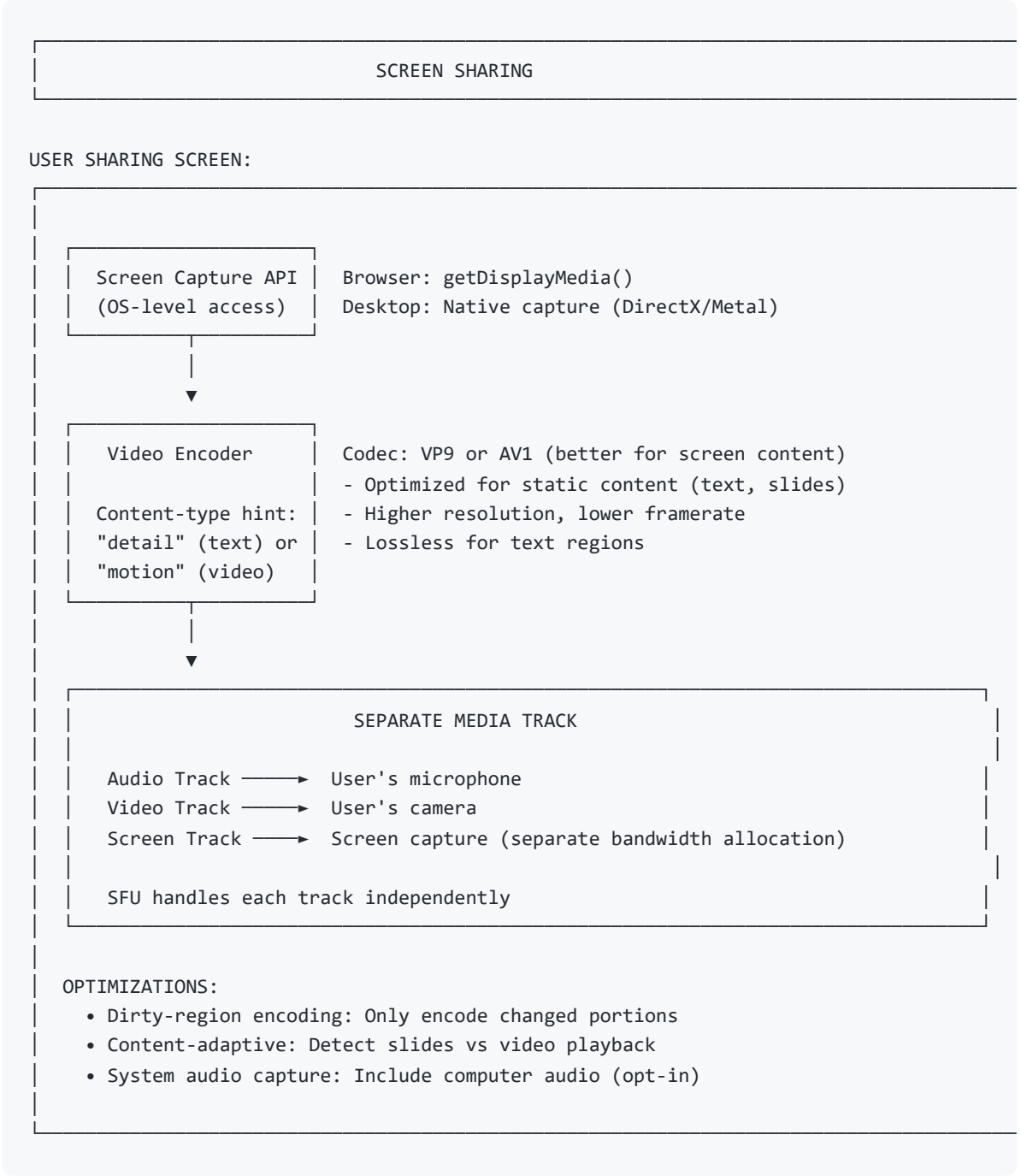
- Medium: 720p @ 1.0 Mbps
- Low: 360p @ 0.3 Mbps

SFU selects which layer to forward based on:
- Receiver's available bandwidth
- Receiver's viewport size (is participant thumbnail or spotlight?)
- Active speaker detection (prioritize current speaker)

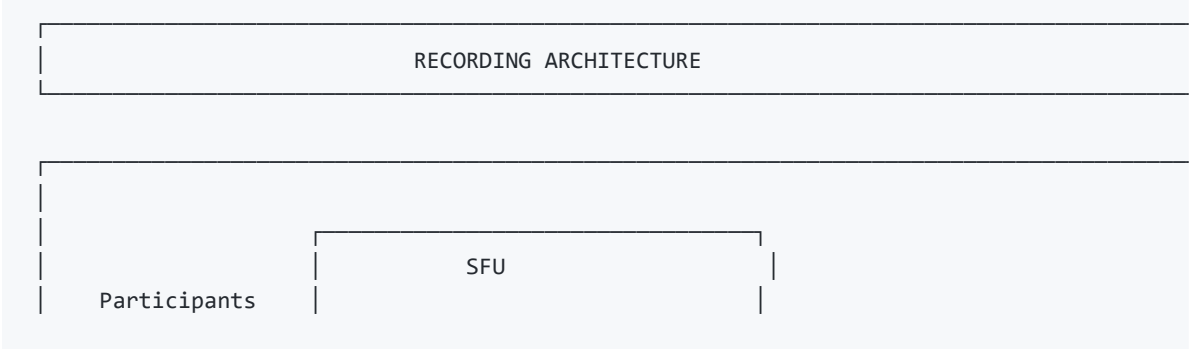
Large Meeting Optimization (1000+ participants)

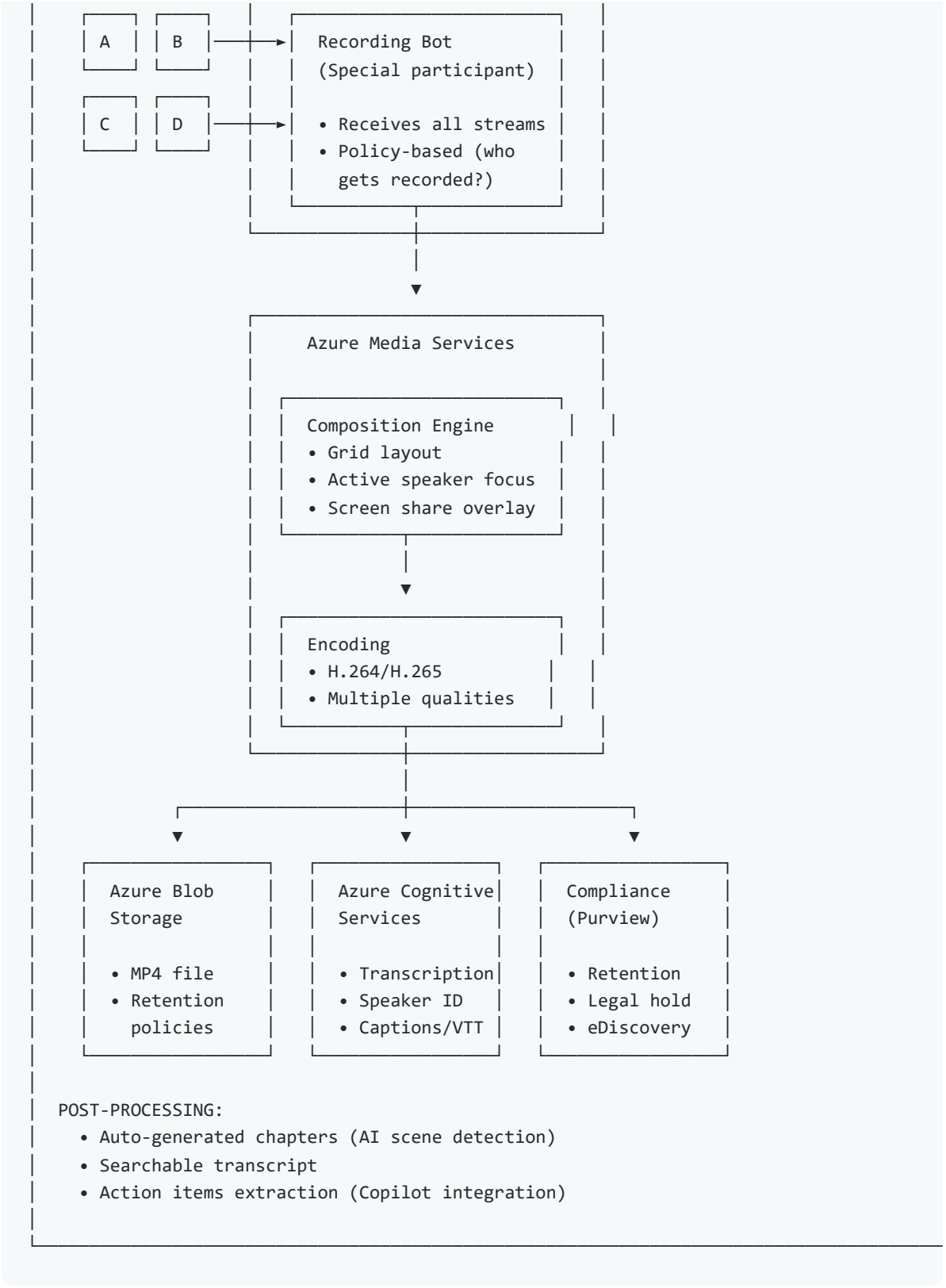


Screen Sharing Architecture



Call Recording Architecture

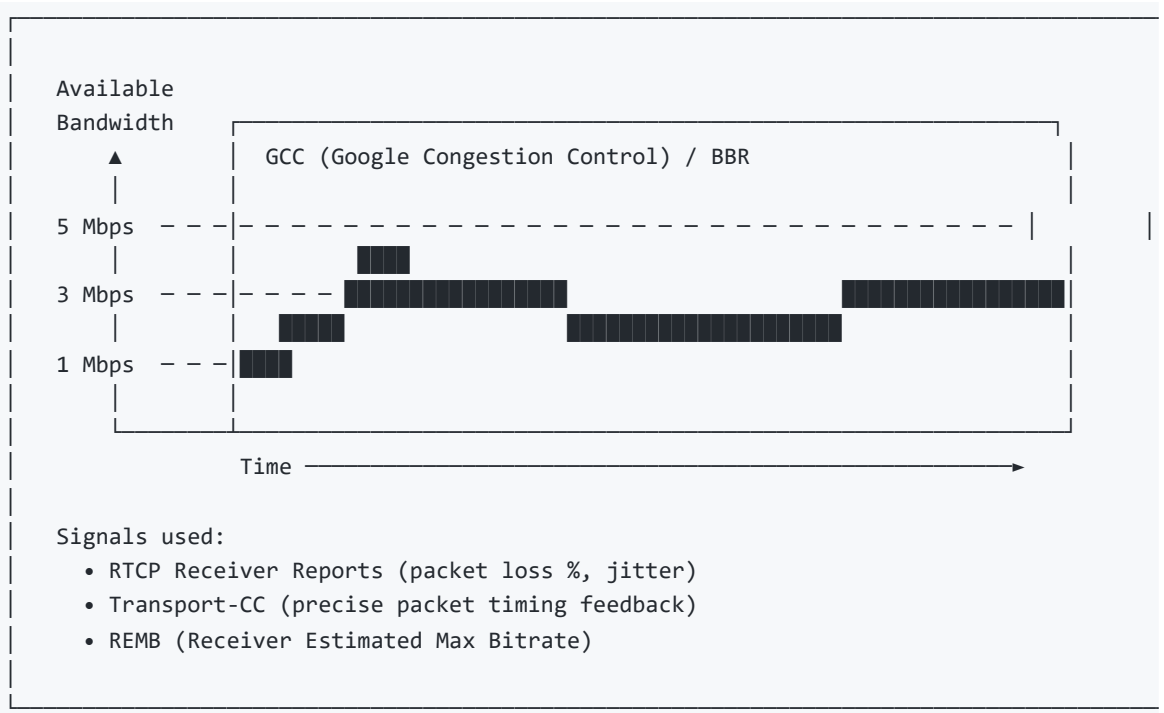




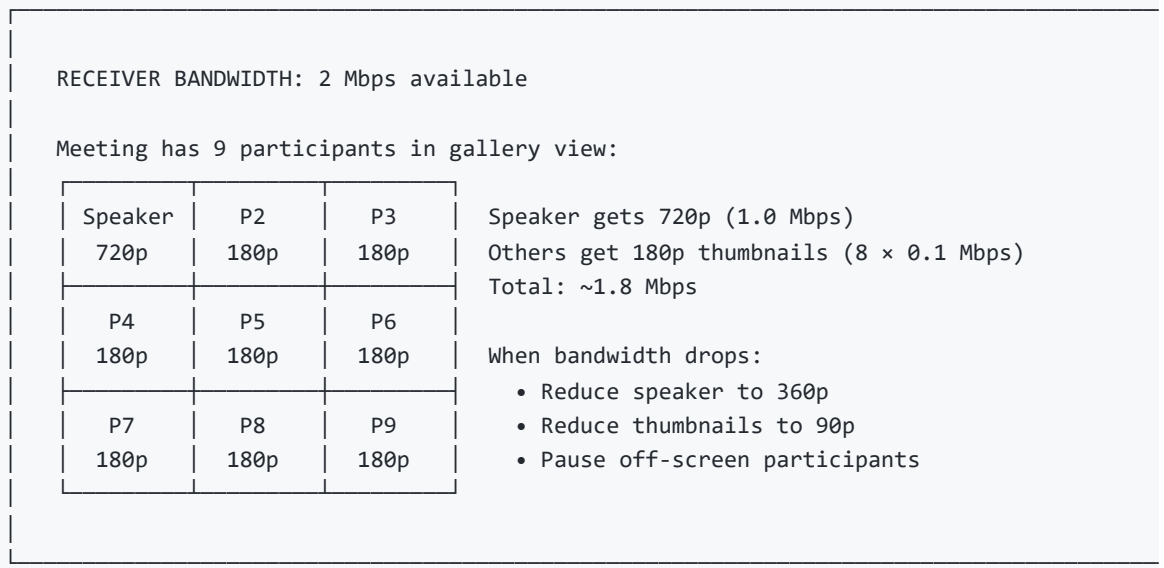
Quality Adaptation (Bandwidth Estimation)

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          ADAPTIVE BITRATE CONTROL                            │
└──────────────────────────────────────────────────────────────────────────────┘

SENDER SIDE (Congestion Control):
```

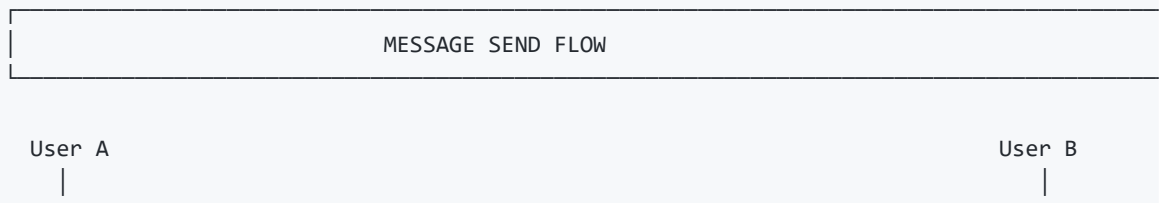


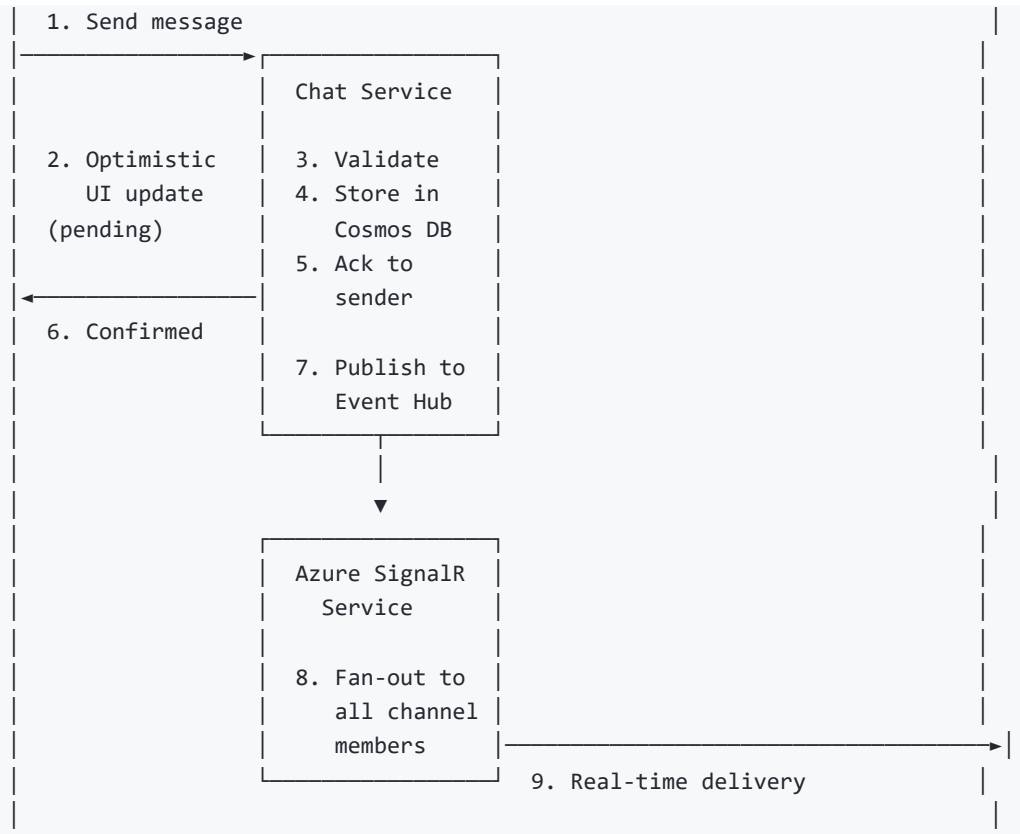
SFU ADAPTATION DECISIONS:



Chat Service Deep Dive

Message Flow





OFFLINE USER HANDLING:

- Message stored in Cosmos DB with delivery_status = 'pending'
- When user comes online, sync service fetches unread messages
- Push notification sent via Azure Notification Hubs

Database Schema

```
-- Cosmos DB document structure (JSON)
{
  "id": "msg_123456789",
  "partitionKey": "channel_abc",  -- Partition by channel for locality
  "type": "message",
  "channelId": "channel_abc",
  "threadId": "thread_xyz",       -- null for top-level messages
  "senderId": "user_123",
  "content": {
    "text": "Hello team!",
    "mentions": ["user_456"],
    "format": "html"
  },
  "attachments": [
    {
      "type": "file",
      "blobUrl": "https://...",
      "name": "report.pdf",
      "size": 1024000
    }
  ],
```

```
    "reactions": {
      "👍": ["user_456", "user_789"],
      "❤️": ["user_456"]
    },
    "createdAt": "2024-01-15T10:30:00Z",
    "editedAt": null,
    "deletedAt": null,
    "_ts": 1705314600  -- Cosmos DB timestamp for TTL/sync
}

-- Azure SQL for relational data
CREATE TABLE Teams (
    id UNIQUEIDENTIFIER PRIMARY KEY,
    name NVARCHAR(255),
    tenant_id UNIQUEIDENTIFIER,
    created_at DATETIME2,
    settings NVARCHAR(MAX)  -- JSON
);

CREATE TABLE Channels (
    id UNIQUEIDENTIFIER PRIMARY KEY,
    team_id UNIQUEIDENTIFIER FOREIGN KEY REFERENCES Teams(id),
    name NVARCHAR(255),
    type VARCHAR(20),  -- 'standard', 'private', 'shared'
    created_at DATETIME2
);

CREATE TABLE ChannelMembers (
    channel_id UNIQUEIDENTIFIER,
    user_id UNIQUEIDENTIFIER,
    role VARCHAR(20),  -- 'owner', 'member'
    joined_at DATETIME2,
    PRIMARY KEY (channel_id, user_id)
);
```

Key Components Summary

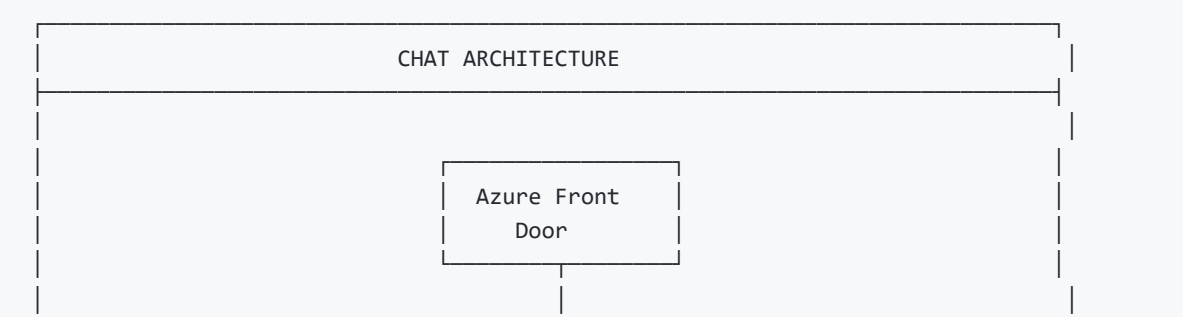
Component	Purpose	Azure Service	Why This Choice
Global Load Balancer	Route to nearest region	Azure Front Door	Anycast, WAF, SSL
API Gateway	Auth, rate limiting	Azure API Management	Enterprise features
Compute	Run microservices	AKS (Kubernetes)	Auto-scale, density
Real-time Messaging	WebSocket management	Azure SignalR Service	100K connections/unit

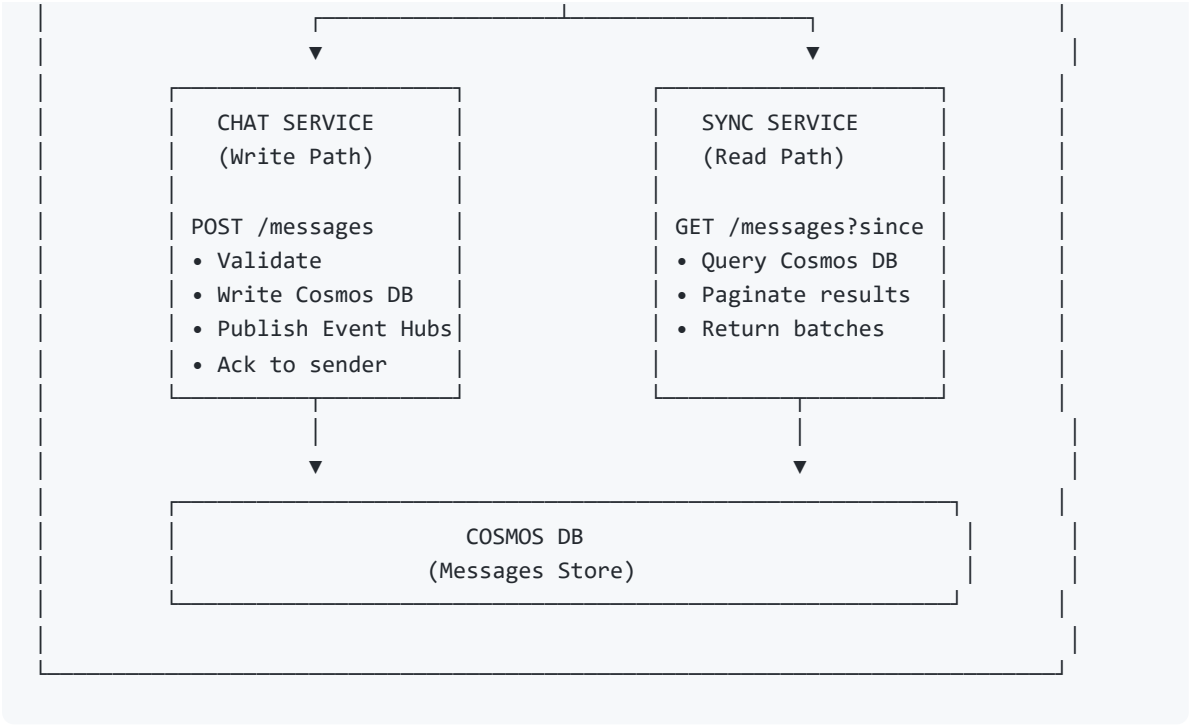
Message Queue	Async processing	Azure Event Hubs	Kafka API, partitioned
Task Queue	Reliable job queue	Azure Service Bus	Dead letter, retry
Chat Storage	Messages, threads	Cosmos DB	Multi-region writes
Relational Data	Users, teams, orgs	Azure SQL Database	ACID, joins
Cache	Sessions, presence	Azure Cache for Redis	Sub-ms latency
File Storage	Attachments, recordings	Azure Blob Storage	Tiered, cheap
Search	Message search	Azure AI Search	Full-text, faceted
Video/Calls	WebRTC infrastructure	Azure Comm. Services	SFU, TURN included
Recording	Call recording	Azure Media Services	Encode, stream
Identity	SSO, MFA	Azure AD / Entra ID	Enterprise SSO
Secrets	Keys, certs	Azure Key Vault	HSM-backed
Monitoring	Logs, metrics	Azure Monitor	Unified observability
Compliance	Data governance	Azure Purview	GDPR, eDiscovery

Feature Flows

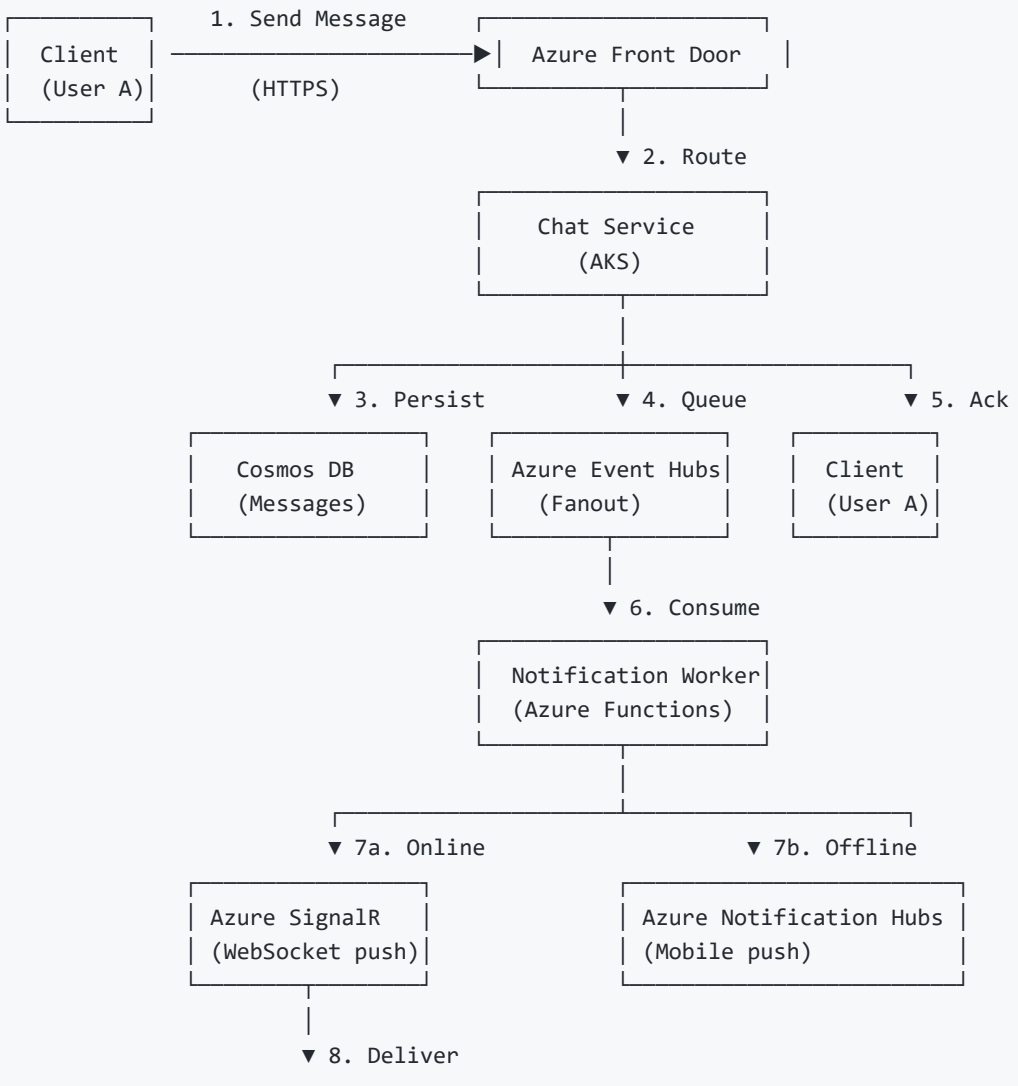
1. Chat Message Flow

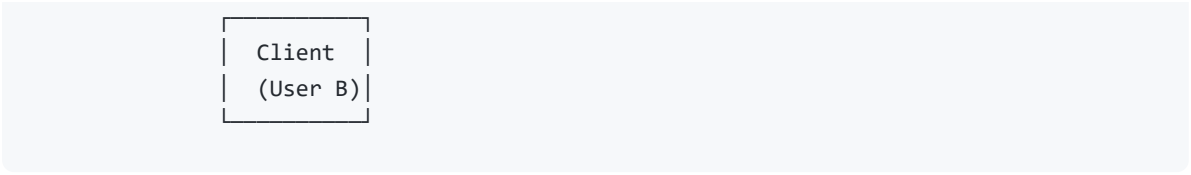
Two Paths: Write (Send) vs Read (Sync)





Write Path: Sending a Message (Real-time)



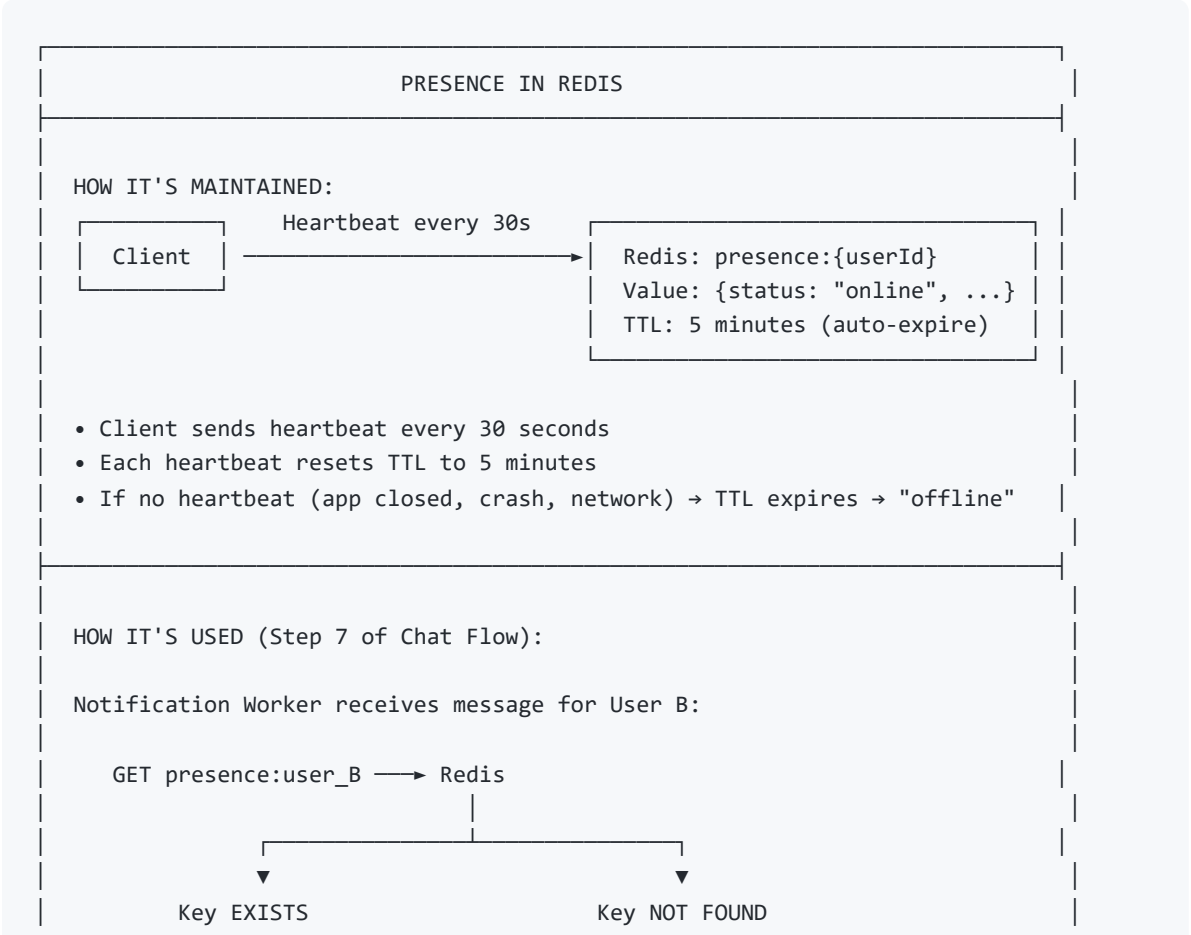


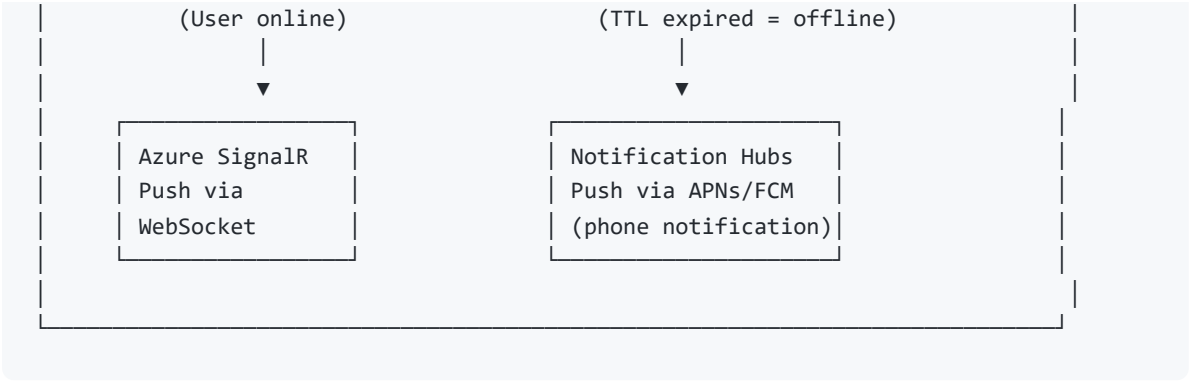
Write Path Steps:

1. User A sends message via HTTPS
2. Front Door routes to Chat Service in AKS
3. Message persisted to Cosmos DB (durable)
4. Message published to Event Hubs for fanout
5. Client A gets acknowledgment (message sent ✓)
6. Azure Functions worker consumes from Event Hubs
7. Check recipient status in Redis → route message:
   - **Online** → Push via SignalR WebSocket
   - **Offline** → Push via Notification Hubs (mobile)
8. User B receives message in real-time

Presence: How Step 7 Works

Presence determines **how to deliver** the message. It's stored in Redis and checked at delivery time.





Why TTL-based presence?

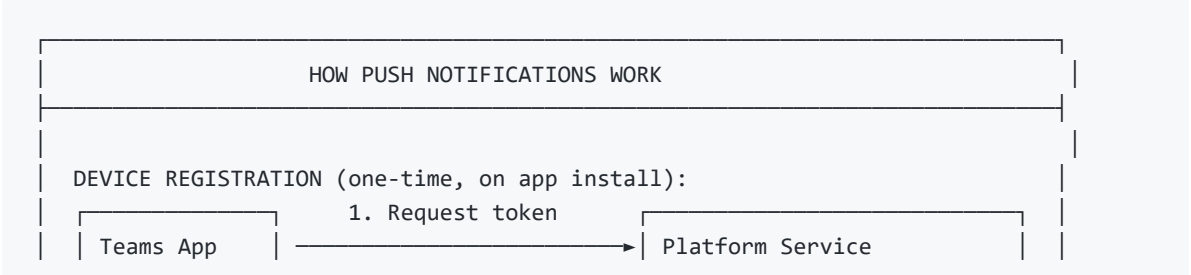
Approach	Problem
Explicit "disconnect" event	Unreliable - crashes, network loss don't trigger it
TTL + heartbeat ✓	Auto-expires; handles all failure modes

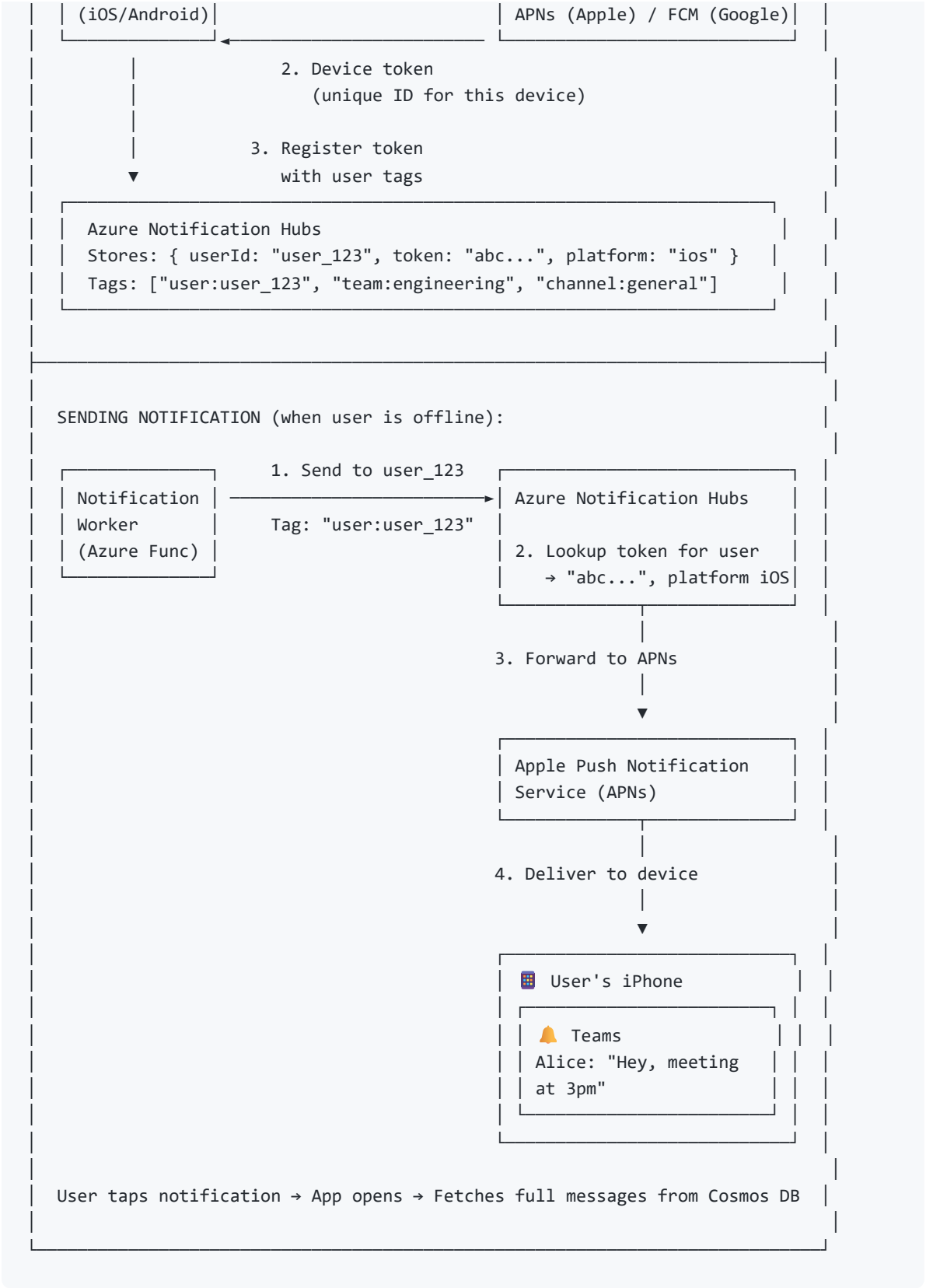
Design Decisions Explained:

Decision	Why This Choice	Alternative Considered
Event Hubs for fanout	High throughput (millions/sec), partitioned by channelId, Kafka-compatible	Service Bus - better for task queues but lower throughput
Cosmos DB first, then queue	Durability - message is safe even if fanout fails; can replay from DB	Queue first risks message loss if DB write fails
Azure Functions as worker	Serverless, auto-scale with Event Hubs partitions, cost-efficient	AKS pods - more control but manual scaling
Redis TTL for presence	Auto-cleanup on crash/disconnect; no explicit "offline" event needed	Explicit disconnect - unreliable

Azure Notification Hubs: Reaching Offline Users

When a user is **offline** (app closed, phone locked, no WebSocket), we can't push via SignalR. Instead, we use **platform push notifications** (the same system that shows "You have a new email" on your lock screen).





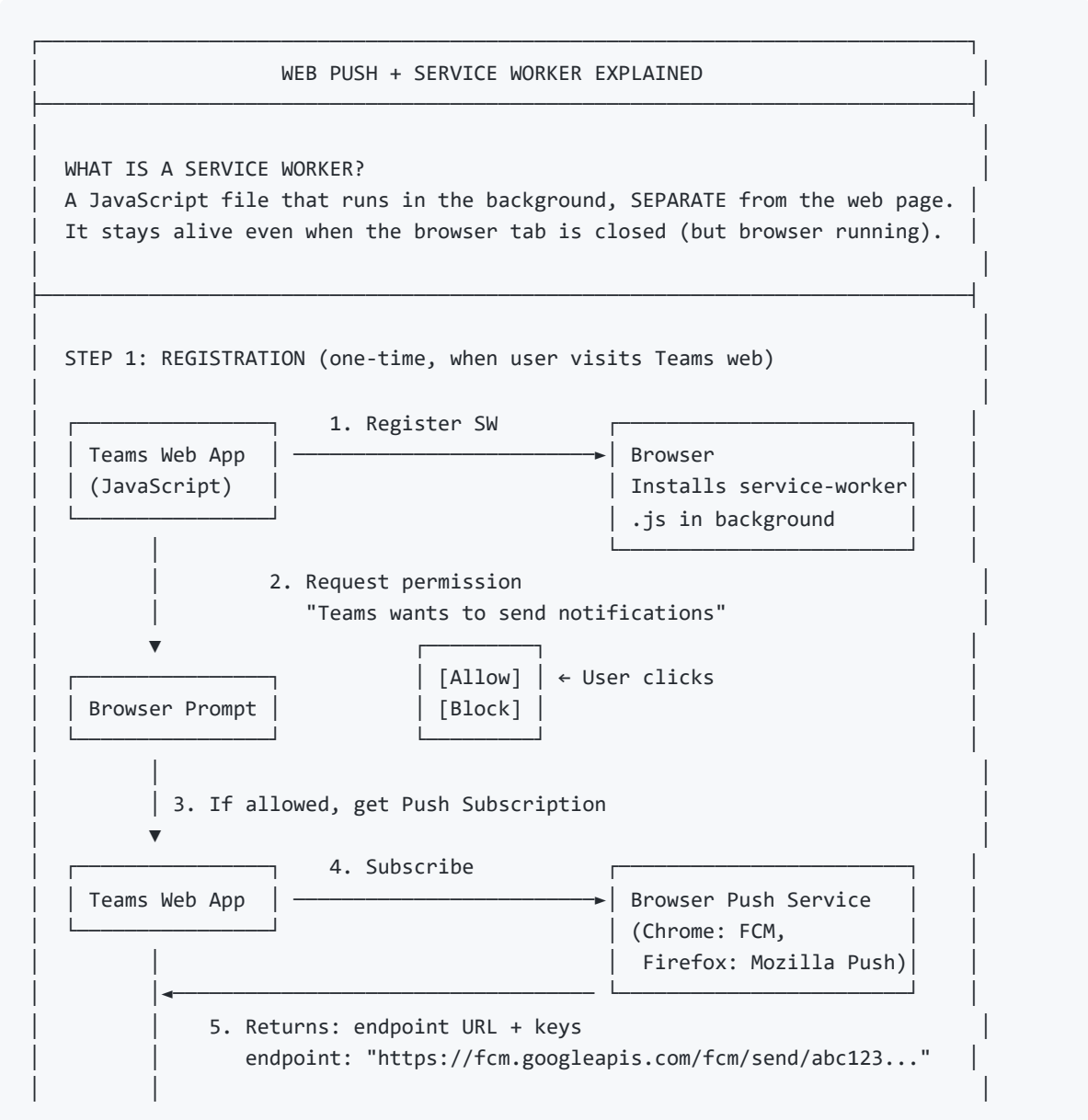
Notification Delivery by Platform:

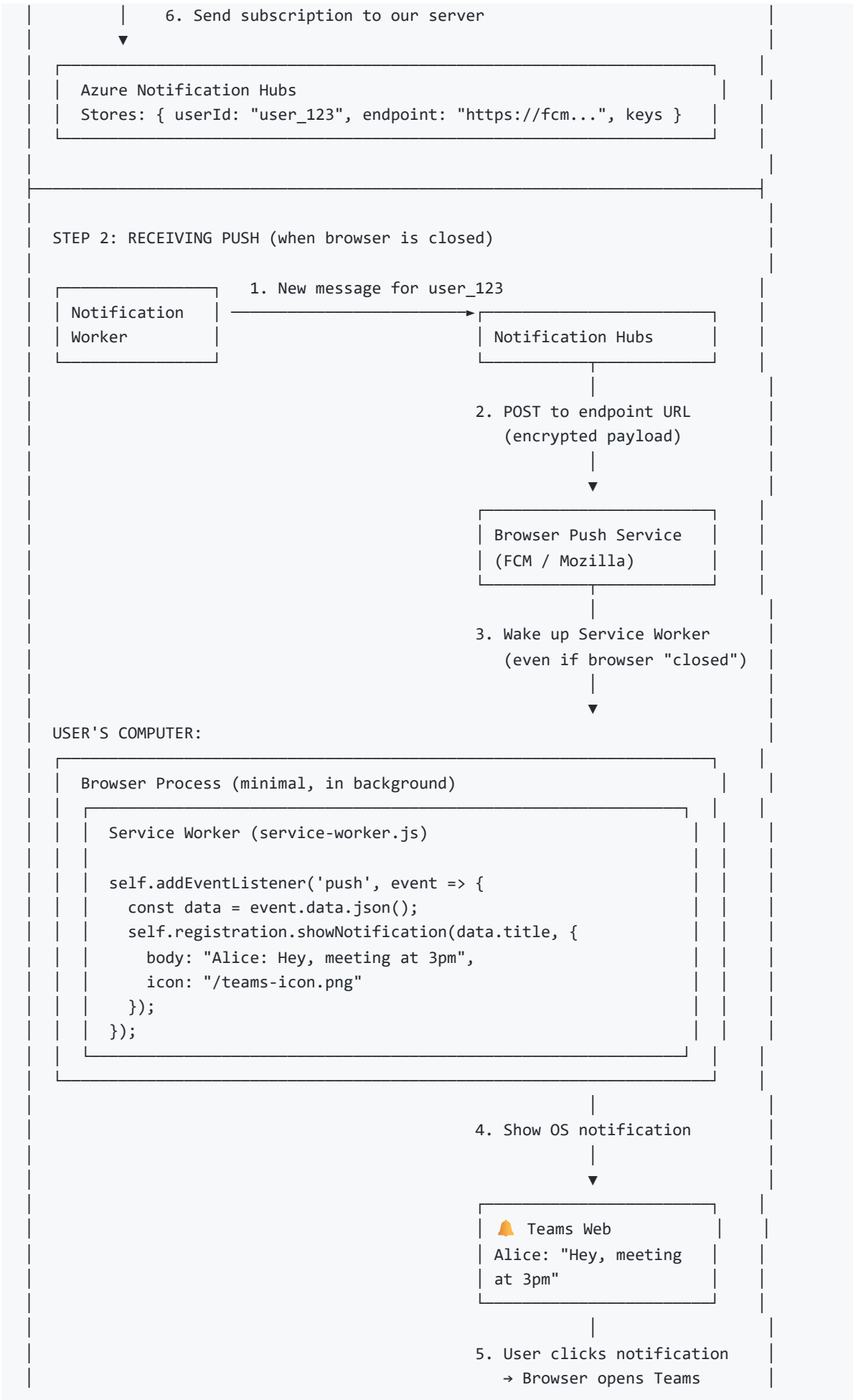
Platform	Push Service	How it works

iOS	APNs (Apple Push Notification)	Works even when app killed
Android	FCM (Firebase Cloud Messaging)	Works even when app killed
Windows Desktop	WNS (Windows Notification Service)	Toast notifications via Action Center
Mac Desktop	APNs (same as iOS)	Native macOS notifications
Web Browser	Web Push API + Service Worker	Works if browser open OR user allowed background

Web & Desktop: Key Differences from Mobile

How Web Push + Service Worker Works:




6. Send subscription to our server
Azure Notification Hubs
Stores: { userId: "user_123", endpoint: "https://fcm...", keys }
STEP 2: RECEIVING PUSH (when browser is closed)
Notification Worker
1. New message for user_123
Notification Hubs
2. POST to endpoint URL
(encrypted payload)
Browser Push Service
(FCM / Mozilla)
3. Wake up Service Worker
(even if browser "closed")
USER'S COMPUTER:
Browser Process (minimal, in background)
Service Worker (service-worker.js)
self.addEventListener('push', event => {
const data = event.data.json();
self.registration.showNotification(data.title, {
body: "Alice: Hey, meeting at 3pm",
icon: "/teams-icon.png"
});
});
4. Show OS notification
Teams Web
Alice: "Hey, meeting at 3pm"
5. User clicks notification
→ Browser opens Teams

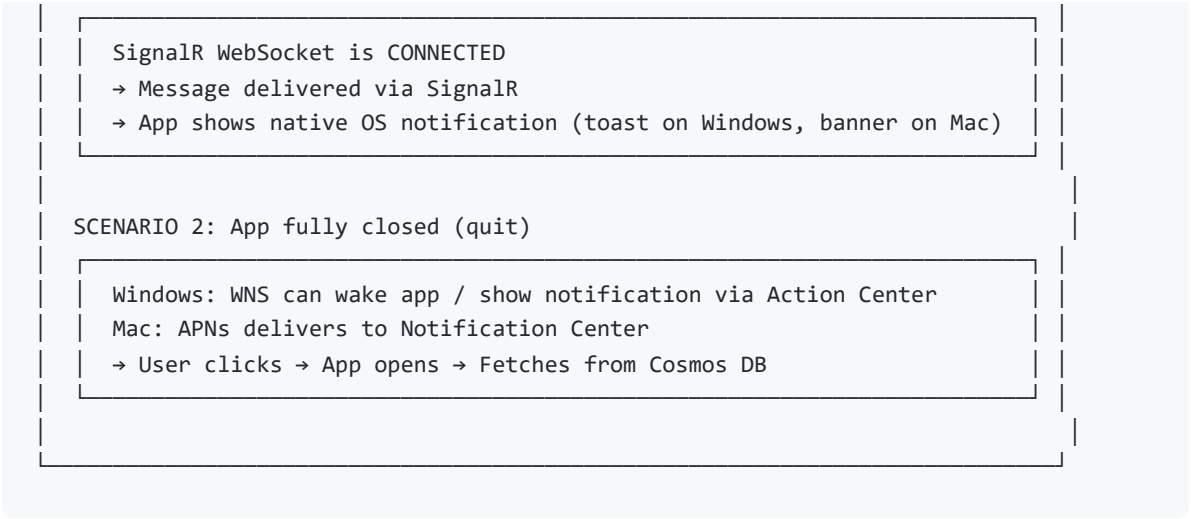
```
│                                                    → Fetches from Cosmos DB    │
│                                                                                │
└────────────────────────────────────────────────────────────────────────────────┘
```

Limitations of Web Push:

Limitation	Impact
User must grant permission	Many users click "Block"
Browser must be installed	Obvious, but matters
Some browsers need to be running	Safari on Mac needs browser running
Doesn't work in Incognito	No Service Worker persistence
iOS Safari: No Web Push (until iOS 16.4+)	Major gap for iPhone web users

Web Browser Notification Scenarios:

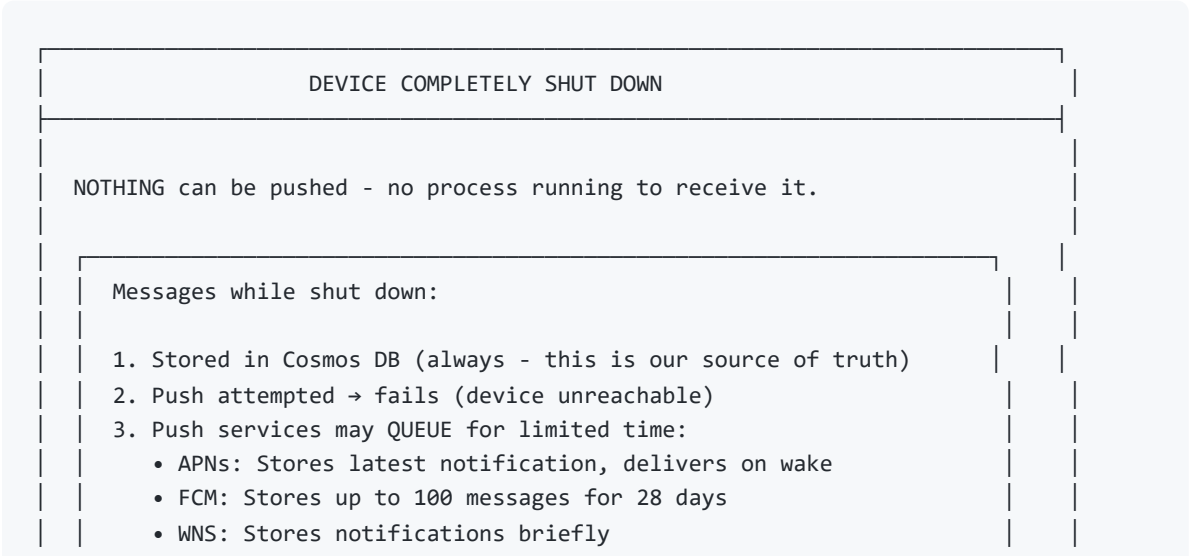
```
┌──────────────────────────────────────────────────────────────────────────────┐
│                        WEB BROWSER NOTIFICATIONS                             │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  SCENARIO 1: Browser tab open (background tab)                               │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │  SignalR WebSocket is STILL CONNECTED                                │    │
│  │  → Message delivered via SignalR (same as "online")                  │    │
│  │  → Browser shows notification via Notification API                   │    │
│  └──────────────────────────────────────────────────────────────────────┘    │
│                                                                              │
│  SCENARIO 2: Browser closed, but Service Worker registered                   │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │  User previously granted notification permission                     │    │
│  │  → Web Push via Service Worker (browser wakes up in background)      │    │
│  │  → Shows OS-level notification                                       │    │
│  │  → User clicks → Browser opens → Fetches from Cosmos DB              │    │
│  └──────────────────────────────────────────────────────────────────────┘    │
│                                                                              │
│  SCENARIO 3: Browser closed, no Service Worker / permission denied           │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │  ❌ Cannot push notification                                         │    │
│  │  → Message stored in Cosmos DB                                       │    │
│  │  → User sees it when they open Teams next time (pull on reconnect)   │    │
│  │  → May receive notification on mobile device instead                 │    │
│  └──────────────────────────────────────────────────────────────────────┘    │
│                                                                              │
├──────────────────────────────────────────────────────────────────────────────┤
│                        DESKTOP APP NOTIFICATIONS                             │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  SCENARIO 1: App running (foreground or minimized)                           │
```



Summary: Delivery Strategy by State

User State	Delivery Method	Reliability
App open (any platform)	SignalR WebSocket	✅ Instant
App closed - Mobile	APNs/FCM push	✅ Reliable
App closed - Windows	WNS push	✅ Reliable
App closed - Mac	APNs push	✅ Reliable
Browser closed + Service Worker	Web Push	⚠️ Depends on browser/OS
Browser closed, no permission	**No push** - pull on reconnect	❌ User must open app
Device shut down	**No push possible**	❌ Pull on next boot

What happens when device is shut down?



```
│  │  4. When device boots → queued push delivered (if not expired)       │    │
│  │  5. App opens → fetches ALL missed messages from Cosmos DB           │    │
│  │                                                                      │    │
│  └──────────────────────────────────────────────────────────────────────┘    │
│                                                                              │
│  KEY INSIGHT: Push notifications are just ALERTS, not the messages.          │
│  The real messages are ALWAYS in Cosmos DB. Push is best-effort.             │
│                                                                              │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │  User shuts down laptop Friday evening                               │    │
│  │  50 messages arrive over weekend                                     │    │
│  │  User boots laptop Monday morning                                    │    │
│  │                                                                      │    │
│  │  → Maybe 1 push notification shows (APNs keeps latest only)          │    │
│  │  → User opens Teams                                                  │    │
│  │  → App sends lastSeenTimestamp to server                             │    │
│  │  → Server returns all 50 messages from Cosmos DB                     │    │
│  │  → User sees everything they missed                                  │    │
│  └──────────────────────────────────────────────────────────────────────┘    │
│                                                                              │
│  MULTI-DEVICE helps: If laptop is off but phone is on, phone gets push.      │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘
```

Key points:

- Push notification is just a "**wake up" alert** - it contains minimal data (sender, preview)
- Full message content is fetched from Cosmos DB when app opens
- **Cosmos DB is the source of truth** - push is best-effort delivery
- Device tokens are managed by Apple/Google/Microsoft, not us
- Notification Hubs abstracts the complexity of APNs/FCM/WNS APIs
- **Web is the least reliable** for offline push - encourage users to install desktop/mobile app
- **Multi-device**: If one device is off, another may receive the push

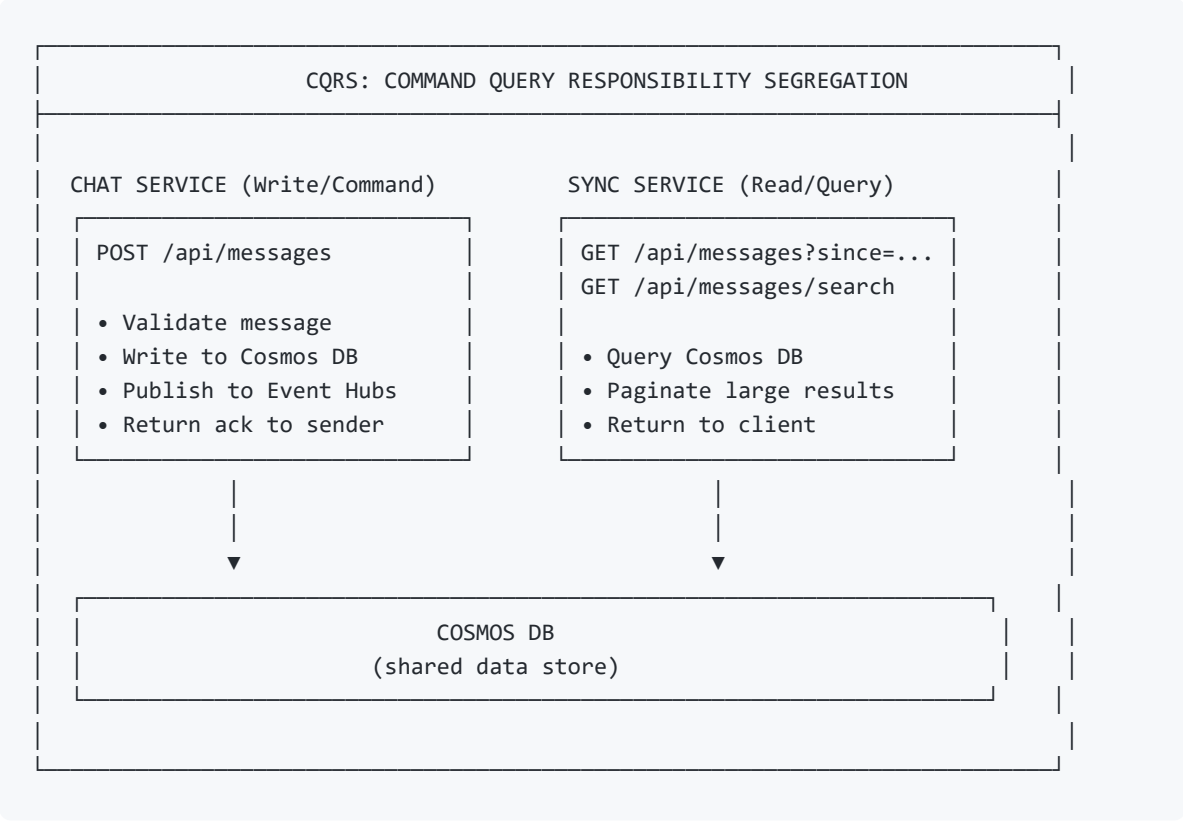
Why Notification Hubs (not direct APNs/FCM)?

Without Notification Hubs	With Notification Hubs
Integrate 3+ APIs (APNs, FCM, WNS, Web Push) separately	Single unified API
Manage device tokens yourself	Automatic token management
Handle token expiry/refresh	Handles automatically
Build retry/failover logic	Built-in reliability
Scale infrastructure yourself	Scales to millions

Offline → Online: Message Sync (Sync Service)

When a user reconnects after being offline, they need to fetch missed messages. This is handled by a **separate Sync Service**, not the Chat Service.

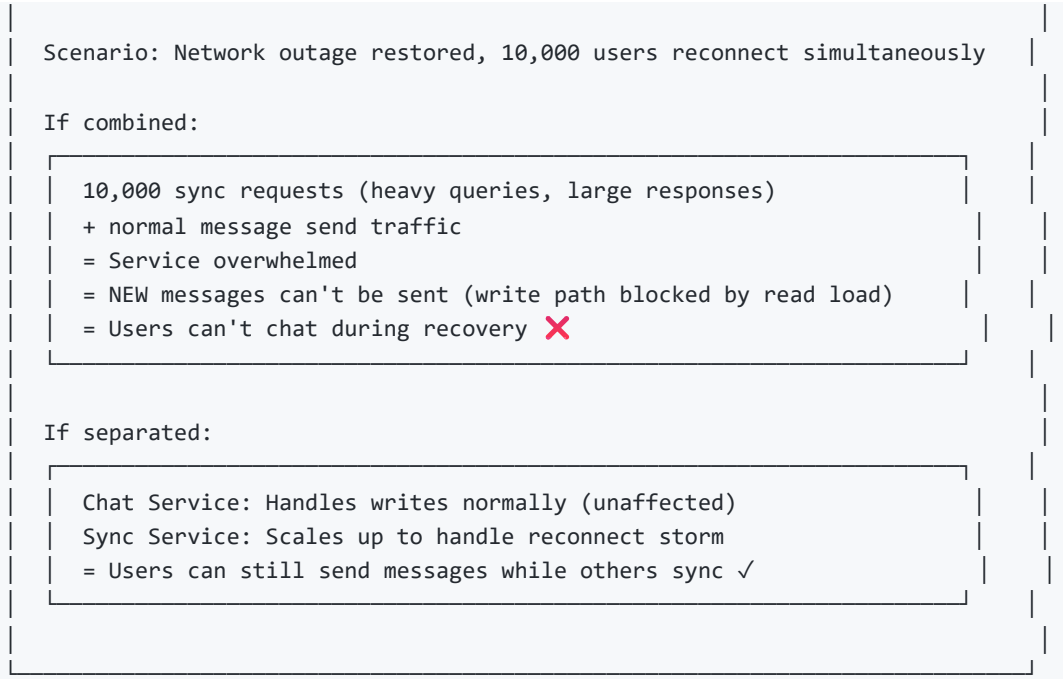
Why Separate Services? (CQRS Pattern)



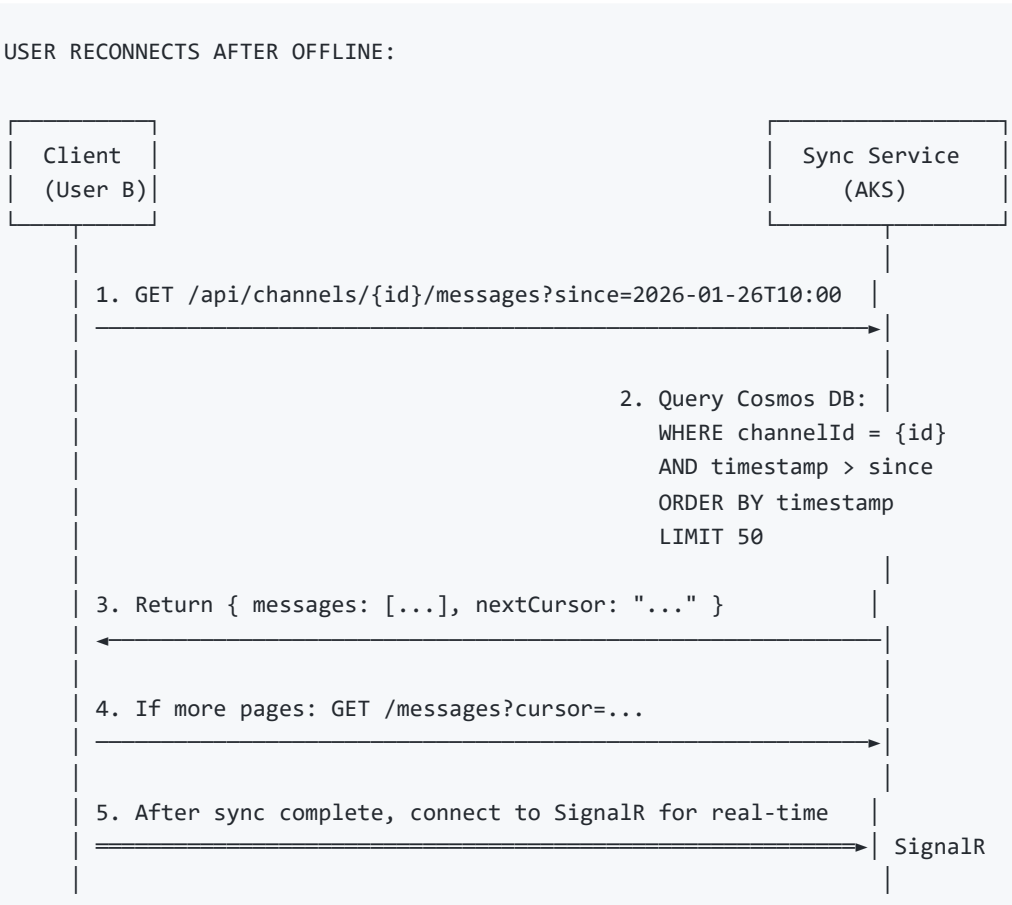
Aspect	Chat Service (Write)	Sync Service (Read)
Pattern	Async, fire-and-forget	Sync, request-response
Operation	Write + publish event	Query + return data
Payload	Small (one message)	Large (hundreds of messages)
Latency	<100ms ack required	Seconds acceptable
Scaling trigger	Messages/second	Concurrent reconnects + data size
Resource needs	CPU (validation, publish)	Memory + IO (large queries)
Failure impact	Message not sent (critical)	User retries (less critical)

Why NOT combine them?





Message Sync Flow:



Sync Strategy:

1. Client stores `lastSeenTimestamp` locally (IndexedDB/SQLite)

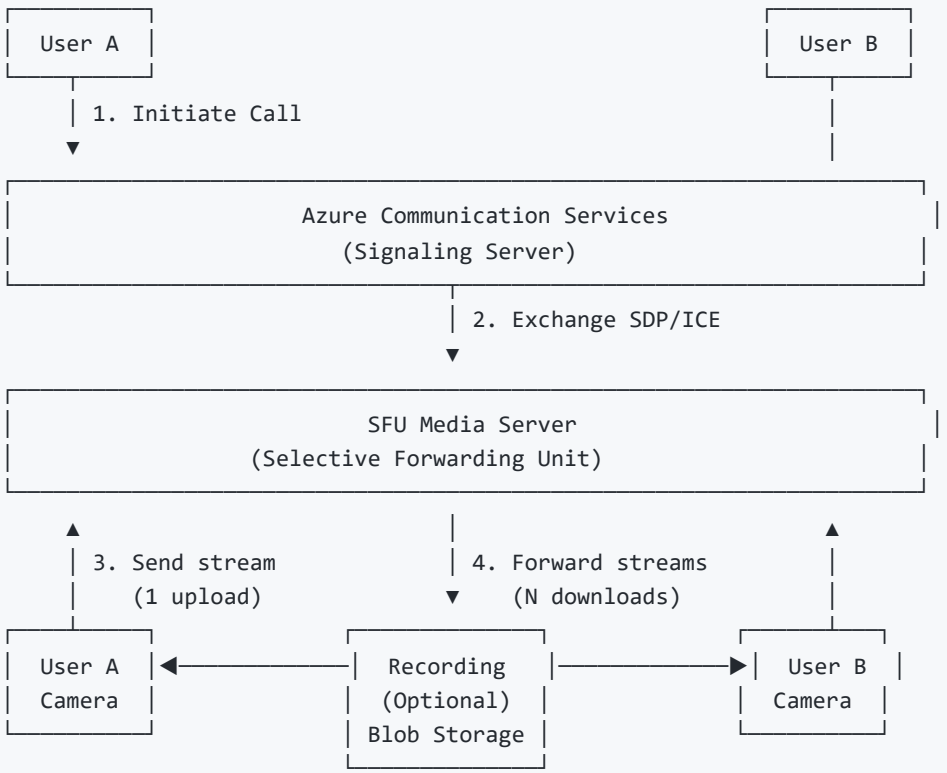
2. On reconnect, call Sync Service with `lastSeenTimestamp`
3. Sync Service queries Cosmos DB for `timestamp > lastSeen`
4. Return in batches (50 msgs) with cursor for pagination
5. After sync complete, subscribe to SignalR for real-time updates

Edge Cases:

- Long offline (days): Paginate, fetch on scroll (don't load all at once)
- Conflicting edits: Server timestamp wins (last-write-wins)
- Deleted messages: Soft-delete flag, client removes from UI
- Multiple devices: Each device tracks own `lastSeenTimestamp`
- Thundering herd: Sync Service auto-scales independently of Chat Service

---

2. Video Call Flow



Steps:

1. User A initiates call → Signaling server notifies User B
2. Exchange SDP offers and ICE candidates for connection setup
3. Each user uploads ONE stream to SFU (simulcast: 1080p/720p/360p)
4. SFU selectively forwards streams to participants based on bandwidth

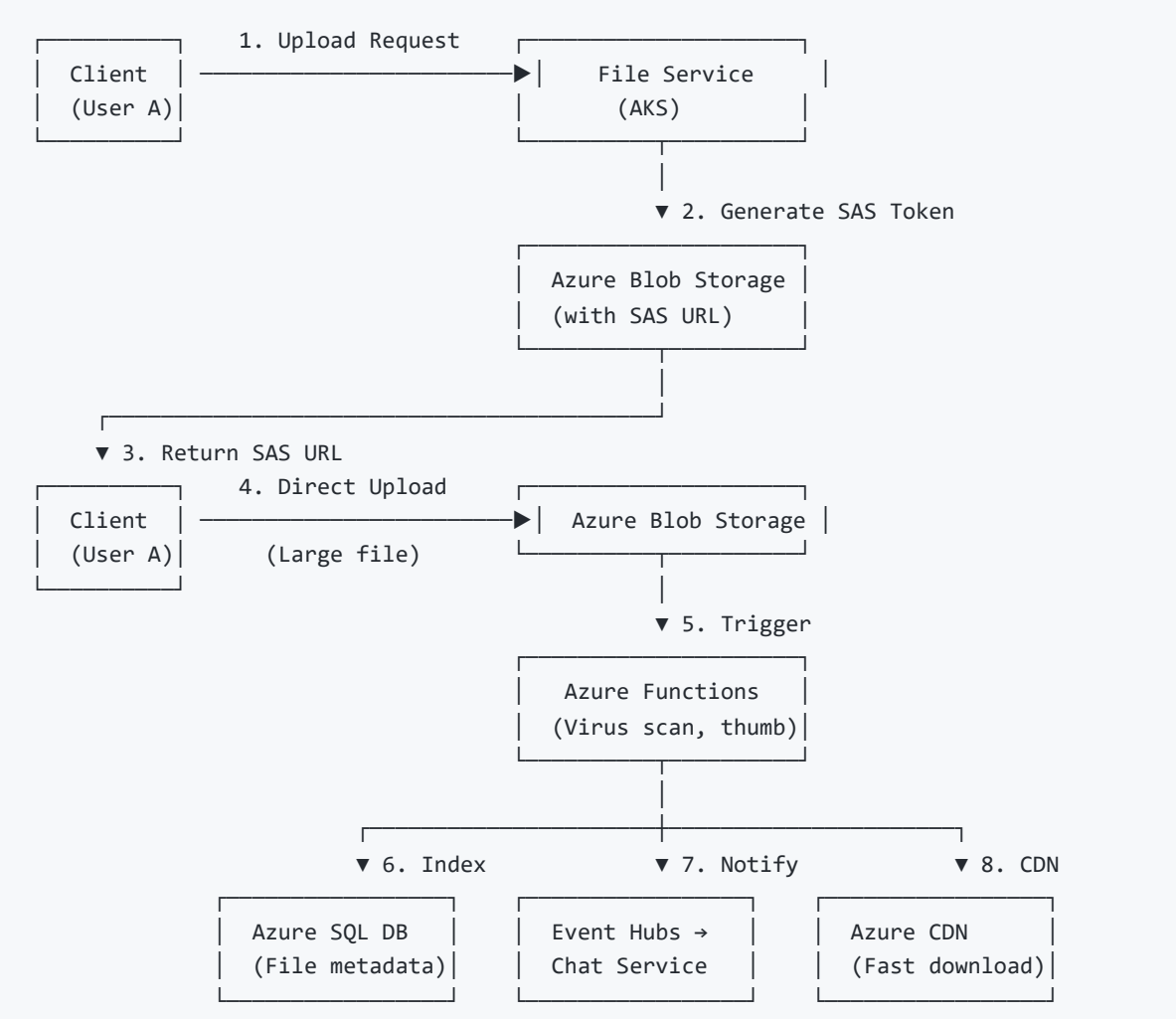
5. Optional: Recording bot captures and stores to Azure Blob Storage

Design Decisions:

Decision	Why This Choice
SFU over Mesh	Mesh = N^2 streams (doesn't scale). SFU = N uploads, selective downloads
SFU over MCU	MCU decodes/re-encodes = high CPU + latency. SFU just forwards
Simulcast (3 qualities)	Sender encodes once at 3 levels; SFU picks per receiver bandwidth
Azure Communication Services	Managed TURN/STUN + SFU, no infrastructure to maintain

---

3. File Sharing Flow



Steps:

1. Client requests upload URL from File Service
2. Service generates time-limited SAS token for Blob Storage
3. Client receives secure upload URL
4. Client uploads directly to Blob Storage (offloads server)
5. Blob trigger fires Azure Function for processing (virus scan, thumbnail)
6. Metadata stored in Azure SQL (filename, size, owner, channel)
7. Notification sent to chat participants via Event Hubs
8. File cached at Azure CDN edge for fast downloads

Design Decisions:

Decision	Why This Choice
SAS token + direct upload	Offloads server; client uploads directly to Blob (scales better)
Blob trigger → Functions	Serverless processing; scales with upload volume
Metadata in SQL, file in Blob	SQL for queries/joins, Blob for cheap large file storage
CDN for downloads	Edge caching; fast delivery globally

Database Configurations

1. Azure Cosmos DB (Messages & Threads)

Setting	Value
Container	`messages`
Partition Key	`/channelId`
Consistency	Session (user sees own writes)
Throughput	Autoscale 400 - 100,000 RU/s
TTL	7 years (compliance) or per-tenant
Multi-region	Enabled (US East, EU West, Asia East)

Document Structure:

```
{
  "id": "msg_uuid",
  "channelId": "channel_123",      // Partition Key
  "senderId": "user_456",
  "content": "Hello team!",
  "timestamp": "2026-01-26T17:00:00Z",
  "threadId": "thread_789",        // For replies
  "type": "text",
  "reactions": [],
  "mentions": ["user_111", "user_222"]
}
```

Why `/channelId` as partition key?

- All messages in a channel queried together
- Even distribution across partitions
- Efficient range queries by timestamp within channel

Why Cosmos DB for messages?

- Write-heavy workload (millions of messages/min)
- Flexible schema (text, files, reactions, edits)
- Multi-region writes for global low latency
- Session consistency = user sees own writes immediately

---

2. Azure SQL Database (Users, Teams, Channels)

Table	Purpose	Key Columns
`Users`	User profiles	`userId (PK)`, `tenantId`, `displayName`, `email`
`Teams`	Team definitions	`teamId (PK)`, `tenantId`, `name`, `createdAt`
`Channels`	Channels in teams	`channelId (PK)`, `teamId (FK)`, `name`, `type`
`TeamMembers`	Team membership	`teamId`, `userId`, `role`, `joinedAt`
`ChannelMembers`	Channel access	`channelId`, `userId`, `role`
`Files`	File metadata	`fileId (PK)`, `channelId`, `blobUrl`, `size`, `uploadedBy`

Indexes:

- `tenantId` on all tables (tenant isolation)
- `teamId` on Channels, TeamMembers
- `channelId` on Files, ChannelMembers

Why SQL for metadata?

- Relational data with joins (user → teams → channels)
- ACID transactions (adding user to team = atomic)
- Complex queries (find all teams user belongs to)
- Read-heavy, infrequent writes

---

3. Azure Cache for Redis (Presence, Sessions, Cache)

Key Pattern	Value	TTL
`presence:{userId}`	`{status, lastSeen, device}`	5 min
`session:{connectionId}`	`{userId, deviceType, region}`	30 min
`typing:{channelId}`	`Set of userIds`	10 sec
`unread:{userId}:{channelId}`	`count`	None
`user:{userId}`	`{profile JSON}`	1 hour

Configuration:

- Cluster: 3 shards with replicas for HA
- Eviction: `allkeys-lru` for cache keys
- Pub/Sub: Used for presence broadcast across instances

Why Redis?

- Sub-millisecond latency for presence checks
- TTL for automatic offline detection
- Pub/Sub for real-time broadcast to all server instances
- Ephemeral data that doesn't need durability

---

4. Azure Blob Storage (Files, Media, Recordings)

Container	Purpose	Access

`attachments`	Chat file attachments	Private (SAS tokens)
`profile-images`	User avatars	Public read via CDN
`recordings`	Meeting recordings	Private (SAS tokens)
`thumbnails`	File/video previews	Public read via CDN
`transcripts`	Meeting transcripts	Private (SAS tokens)

Configuration:

- Lifecycle Policy: Hot → Cool after 30 days, Archive after 1 year
- Soft delete: 14 days retention
- Versioning: Enabled for compliance

Why Blob Storage?

- Cheapest storage for large files (pennies per GB)
- Tiered storage (hot/cool/archive) for cost optimization
- Direct upload via SAS tokens (offloads application servers)
- Integrates with CDN for global delivery

---

5. Azure AI Search (Message & File Search)

Index	Source	Searchable Fields
`messages-index`	Cosmos DB change feed	`content`, `senderId`, `channelId`, `timestamp`
`files-index`	Azure SQL + Blob metadata	`fileName`, `content` (extracted), `uploadedBy`
`people-index`	Azure AD sync	`displayName`, `email`, `department`

Configuration:

- Indexer: Runs every 5 minutes from Cosmos DB change feed
- Security: Filter by `tenantId` in queries
- Features: Fuzzy search, highlighting, autocomplete

Why dedicated Search (not SQL LIKE)?

- Full-text search with ranking and relevance
- Fuzzy matching (typos, synonyms)

- Fast even with millions of messages
- Faceted search (filter by date, sender, channel)

Scalability Considerations

- **Horizontal scaling**: Stateless services in AKS with HPA (Horizontal Pod Autoscaler)
- **Database sharding**: Cosmos DB partitioned by channel_id (messages stay together)
- **Message fanout**: Event Hubs partitioned by team_id, SignalR groups per channel
- **CDN**: Azure CDN for profile pictures, file downloads, static assets
- **Regional deployment**: Active-active in 3+ regions, Cosmos DB multi-master
- **Connection limits**: SignalR service scales to millions of concurrent connections

---

Trade-offs

Decision	Option A	Option B	Recommendation
Message storage	SQL	NoSQL (Cosmos)	**Cosmos** - flexible schema, multi-region
Real-time delivery	Polling	WebSocket	**WebSocket** - low latency
Video architecture	Mesh	SFU	**SFU** - scales to 1000s
Large meetings	SFU only	SFU + CDN broadcast	**Hybrid** - CDN for 100+ attendees
Consistency	Strong	Eventual	**Eventual** - availability > consistency
Recording	Client-side	Server-side	**Server-side** - reliable, compliant
Search	SQL LIKE	Dedicated search	**Azure AI Search** - full-text, fast

---

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
WebSocket	Persistent connections for real-time messaging	Tech Guide
WebRTC	Peer-to-peer video/audio for calls	Tech Guide
Redis	Presence caching, pub/sub for real-time updates	Tech Guide
Cosmos DB	Message storage with global distribution	Tech Guide
Kafka	Event streaming for message ordering	Tech Guide
SignalR	Managed WebSocket service for real-time	Tech Guide
CDN	Static asset delivery, meeting broadcasts	Tech Guide

How to Explain It Well

> **Opening (30 sec)**: "Teams is a real-time collaboration platform. I'll design for 300M DAU supporting chat, video calls up to 1000 participants, and file sharing with 99.99% availability across multiple Azure regions."

> **Structure your answer**:
>
> 1. Clarify requirements (scale, features, compliance)
> 2. Draw high-level architecture with Azure services
> 3. Deep dive into real-time messaging (SignalR + Cosmos)
> 4. Explain video architecture (WebRTC + SFU + simulcast)
> 5. Discuss large meeting optimization (CDN broadcast)
> 6. Address scalability and trade-offs

> **Key talking points**:
>
> - "Real-time messaging uses Azure SignalR Service - managed WebSocket at scale"
> - "Messages stored in Cosmos DB partitioned by channel for locality"
> - "Video uses SFU architecture with simulcast - sender encodes 3 qualities, SFU selects per receiver"
> - "Large meetings (100+) switch to CDN broadcast mode for cost efficiency"
> - "TURN servers handle NAT traversal for ~20% of calls that can't go P2P"
> - "Recording bot joins as participant, streams to Media Services for encoding"

Common Follow-up Questions

1. **How do you handle message ordering?**
    - Cosmos DB timestamp + client-side reordering
    - Event Hubs partitioned by channel ensures order within partition
2. **What if SFU server crashes mid-call?**
    - Client reconnects to different SFU
    - ICE restart negotiates new connection
    - Brief interruption (~2-3 seconds)
3. **How do you prevent echo in calls?**
    - Acoustic Echo Cancellation (AEC) on client
    - WebRTC includes AEC in getUserMedia
4. **How do you handle 1000-person meeting?**
    - Presenters on SFU (bidirectional)
    - Attendees receive via CDN (HLS/DASH broadcast)
    - Q&A/reactions via separate WebSocket channel
5. **How is end-to-end encryption handled?**
    - Currently Teams uses TLS in transit + encryption at rest
    - E2EE for 1:1 calls available (keys never leave client)
    - Group E2EE challenging with SFU (server can't forward what it can't read)